

Hebbian Descent: A Unified View on Log-Likelihood Learning

Jan Melchior

jan.melchior@ini.rub.de

Robin Schiewer

robin.schiewer@ini.rub.de

Laurenz Wiskott

laurenz.wiskott@ini.rub.de

Ruhr University Bochum, 44801 Bochum, Germany

This study discusses the negative impact of the derivative of the activation functions in the output layer of artificial neural networks, in particular in continual learning. We propose Hebbian descent as a theoretical framework to overcome this limitation, which is implemented through an alternative loss function for gradient descent we refer to as Hebbian descent loss. This loss is effectively the generalized log-likelihood loss and corresponds to an alternative weight update rule for the output layer wherein the derivative of the activation function is disregarded. We show how this update avoids vanishing error signals during back-propagation in saturated regions of the activation functions, which is particularly helpful in training shallow neural networks and deep neural networks where saturating activation functions are only used in the output layer. In combination with centering, Hebbian descent leads to better continual learning capabilities. It provides a unifying perspective on Hebbian learning, gradient descent, and generalized linear models, for all of which we discuss the advantages and disadvantages. Given activation functions with strictly positive derivative (as often the case in practice), Hebbian descent inherits the convergence properties of regular gradient descent. While established pairings of loss and output layer activation function (e.g., mean squared error with linear or cross-entropy with sigmoid/softmax) are subsumed by Hebbian descent, we provide general insights for designing arbitrary loss activation function combinations that benefit from Hebbian descent. For shallow networks, we show that Hebbian descent outperforms Hebbian learning, has a performance similar to regular gradient descent, and has a much better performance than all other tested update rules in continual learning. In combination with centering, Hebbian descent implements a forgetting mechanism that prevents catastrophic interference notably better than the other tested update rules. When training deep neural networks, our experimental results suggest that Hebbian descent has better or similar performance as gradient descent.

1 Introduction

Gradient descent is the commonly used optimization algorithm in machine learning, particularly in artificial neural networks and deep learning. Despite its theoretical foundation and the possibility of proving its convergence analytically even for stochastic gradient descent (Robbins & Siegmund, 1985; Saad, 1998), it can still have slow convergence or get stuck in a suboptimal solution, especially when saturating activation functions like sigmoids are used. One reason for slow convergence in neural networks trained with gradient descent is the vanishing gradient problem originally described by Hochreiter (1991) for recurrent neural networks. However, the underlying principle of vanishing gradients is also relevant for nonrecurrent deep neural networks and even shallow networks, as it is characterized by an unwanted downscaling of the error signal during the backpropagation stage by small values of the derivative of saturating activation functions. In that situation, the resulting parameter updates would be small in magnitude even if the error signal was still large.

While it is apparent that this effect becomes more severe with increasing network depth, a variety of related work and our experiments demonstrate that it matters even for shallow networks. In the context of shallow networks, the potentially negative effect of saturating activation functions has been discussed from various angles. From the perspective of backpropagation, the famous perceptron (Rosenblatt, 1958) algorithm, which uses a step function as a nonlinearity, had its derivative removed during parameter update as the gradient would be zero otherwise. While a good overview of perceptron learning algorithms and its variants using the step activation function is presented by Gallant (1990), the systematic connection between the perceptron learning rule and performing backpropagation without the derivative of the output activation function was not discussed. Biehl and Schwarze (1995) investigated the online learning behavior of single-layer networks with continuous valued outputs and noted formal similarity between the Hebb rule and the backpropagation weight update. While this similarity is not further discussed in their work, we refer to it more formally and in more detail in section 3.1. When using sigmoid output activations, Hinton (1989) proposed to use the cross-entropy as a loss function instead of the squared error, in which case the derivative of the sigmoid output units cancels out in the gradient, which explains why the cross-entropy loss is the preferred loss when training deep networks with sigmoid (and also softmax) outputs today.

In multilayer neural networks, the potentially negative effect of saturating activation functions during backpropagation can accumulate and is then referred to as vanishing gradients. To address the vanishing gradient problem in deep neural networks with sigmoid activation functions, Fahlman (1988) added small values to the gradients if they came from the saturated regime of the activation function and found this to be effective

in some applications. Chen (1990) proposed the heuristics of simply ignoring the derivative of the output-layer activation function during backpropagation, and Ng et al. (1999) and Ooyen and Nienhuis (1992) presented variations of the activation function to mitigate the negative effect of small derivatives during backpropagation. Lee et al. (1993) noted that slow training convergence can be caused by prematurely saturating sigmoids and found a parameter initialization scheme that alleviates the problem. Vitela and Reifman (1993) and Ng et al. (2003) proposed a modification of the derivative during the backward pass of deep networks with sigmoid activation functions that magnified small gradients to speed up learning. Motivated by technical benefits, Hertz et al. (1997) proposed an algorithm that uses an approximation to get rid of the derivative of the activation function during the backward pass. They also mentioned that for the hyperbolic tangent as an output activation function, a loss similar to the cross entropy exists that cancels out the derivative of the activation function. Yu et al. (2002) replaced the sigmoid with a hyperbole tangent activation function with learnable slope that produces larger gradients than the former close to the limit output values of 0 and 1. Following a different paradigm, activation functions with a constant derivative of one in the positive domain have been proposed. Examples of such functions include the rectifier (Fukushima & Miyake, 1982; Hahnloser et al., 2000; Hahnloser & Seung, 2001) or exponential linear unit (Clevert et al., 2015). However, due to their distinctively different mapping behavior, using nonsaturating activation functions, particularly in the output layer, might not be possible depending on the application. A good overview of contributions to improve convergence speed and performance of deep neural networks can be found in Vora and Yagnik (2013) and Kumar et al. (2015).

Although we do not discuss recurrent neural networks (RNNs) in this letter, they play an important role in the history of the vanishing (and exploding) gradient problem and should be mentioned here. In RNNs, vanishing gradients were a major problem until the proposal of long short-term memory cells by Hochreiter and Schmidhuber (1997). During backpropagation, these cells allow for a linear error flow back in time, which helps to overcome the problem. As an alternative, Pascanu et al. (2013) proposed gradient clipping to alleviate exploding gradients and a special regularization term on the error signal to alleviate vanishing gradients in recurrent neural networks. Mikolov et al. (2014) approached the problem of vanishing gradients by encouraging a specific structure of the recurrent weight matrix. An overview of milestone contributions to the field of recurrent neural networks, including measures against the vanishing gradients problem, is in Salehinejad et al. (2018). However, those techniques may impose constraints on network parameters, require the use of a restricted set of activation functions, or build on the recurrent nature of the network. Thus, while drawing inspiration from the various approaches listed above, better strategies for nonrecurrent neural networks may be found.

Another intensively explored approach for addressing the vanishing gradient problem is to take previous parameter updates into account, for example, using a momentum term like in RSprop (Riedmiller & Braun, 1993), Adagrad (Duchi et al., 2011), or Adam (Kingma & Ba, 2014). However, while having similar effects on the vanishing gradient problem, those approaches work quite differently from Hebbian descent. They may even be combined with it (see section 5.2), which is why we consider them as separate techniques outside the scope of this work. They also come with their own drawbacks—for example, including the previous parameter updates in continual learning can be counterproductive since it may slow the network’s reaction to new inputs.

The continual learning setting is particularly challenging for artificial neural networks. Here, a steady stream of data is presented sample by sample or minibatch by minibatch to the network without repetition of samples. A good continual learning algorithm has to balance two partially competing goals. While it should learn efficiently from new data, it should at the same time not forget relevant information about older data. Because of limited memory capacity (i.e., trainable parameters of the learner), eliminating forgetting altogether is hard to achieve in practice. It might even be undesirable to do so as forgetting can implicitly serve as a filter to get rid of outdated information in continual learning (LeCun et al., 2012; Eiter & Kern-Isberner, 2019). Thus, a balance between adopting new information and forgetting about old data is required. Contrary to that, catastrophic interference (McCloskey & Cohen, 1989; Ratcliff, 1990) in continual machine learning refers to the problem of abrupt forgetting, that is, exponentially decreasing performance on past data. It represents a forgetting behavior where old information is disregarded by the learner too fast. To overcome catastrophic interference, several approaches have been proposed. French (1991) used sparse hidden representations, which reduce interference. Rehearsal learning (Robins, 1995) is a popular approach but requires storing all previously seen patterns. Pseudo-rehearsal learning (Robins, 1995) calculates the output of random patterns and updates the network on a new pattern and several random input-output pairs to reduce interference but adds a significant computational overhead. Complementary learning systems (McClelland et al., 1995; Ans & Rousset, 1997; French, 1997) use a fast and a slowly learning network to store recent and all patterns, respectively. The fast learning network acts as a buffer that rapidly stores recent patterns, which are then carefully transferred to the slowly learning network, which tries to store not only the latest but all patterns. A disadvantage of such systems is that we need two networks for a task that can potentially be solved by a single network and that the knowledge transfer from the fast to the slowly learning network needs consolidation (rehearsal) again. Elastic weight consolidation (Kirkpatrick et al., 2017) regularizes weights toward previous values and removes the need for rehearsal, but requires storing all previous weights. Memory-augmented neural networks

(Santoro et al., 2016) perform continual learning but without a neural implementation of the memory. In conclusion, implementing continual learning using gradient-based methods in a neural network is still a challenging problem.

A learning rule that can perform continual learning of uncorrelated patterns is Hebbian learning, which remains the major learning principle since Donald Hebb postulated his theory in 1949 (Hebb, 1949). It is still widely used in its canonical form generally known as Hebb's rule, which, however, cannot learn negative or inhibitory weights when assuming positive firing rates. The covariance rule (Sejnowski & Tesauro, 1989) was proposed as an alternative to overcome this limitation. An advantage of both learning rules is that they are capable of continual learning, allowing patterns to be stored instantaneously without need for repetition. A disadvantage is that they do not take advantage of seeing input patterns several times. They also have problems with correlated patterns as Marr et al. (1991) stated and Löwe et al. (1998) and Neher et al. (2015), respectively analyzed for autoassociative (unsupervised) and hetero-associative (supervised) networks. Furthermore, Hebb's rule and the covariance rule are unstable learning rules, so that the weights are usually renormalized after each update or a weight decay term is added (Bienenstock et al., 1982), where the latter introduces an additional hyperparameter that controls the speed of forgetting. In unsupervised learning, the stability problem was analytically addressed for a linear neuron by Oja's rule (Oja, 1982) or for several linear neurons by Sanger's rule (Sanger, 1989), which are convergent learning rules that drive the neurons to learn the principal components of the input patterns. For autoassociative learning in Hopfield networks, contrastive Hebbian learning (Rumelhart, McClelland, et al., 1986) and for Boltzmann machines contrastive divergence (Hinton, 2002) and its variants (Tieleman, 2008; Desjardins et al., 2010; Cho et al., 2010) have been proposed as stable learning rules, respectively.

In what follows, we begin with a recapitulation of gradient descent and the concept of centering in artificial neural networks in section 2 and then investigate learning in artificial neural networks without derivatives of activation functions in the output layer. This leads us to a unified view of various well-known algorithms, which we refer to as Hebbian descent. Notice that in the case of the mean squared error (MSE) loss and particular activation functions, the proposed learning rule has previously been discussed in the context of generalized linear models (Nelder & Baker, 1972; Kakade et al., 2011; Goel et al., 2018), contrastive Hebbian learning (Rumelhart, McClelland, et al., 1986; Movellan, 1991; Xie & Seung, 2003), and gradient descent (Rosenblatt, 1958; Hinton, 1989; Hertz et al., 1997). Hebbian descent, however, generalizes this update rule to arbitrary loss and activation functions and shows that if the derivative of the activation function is strictly positive, it is equivalent to performing gradient descent using an alternative loss function named the Hebbian descent loss (see section 3).

Section 3.1 discusses the problem of vanishing activation function derivatives and how Hebbian descent is able to overcome this issue. In section 3.2, we highlight the connections between Hebbian descent and Hebbian learning. In section 3.3, we show that in the case of the MSE loss and an invertible and integrable activation function, Hebbian descent actually optimizes a generalized linear model. As a consequence, the Hebbian descent loss can be seen as the general log-likelihood loss in this case (see appendix B).

After establishing the theoretical connections between these approaches, section 4 describes the experiments performed to compare them. Specifically, we pit Hebbian descent against regular gradient descent, Hebb's rule, and the covariance rule using one-layer networks. The results are described in section 5, where we show that all learning rules considered in this work benefit from centering (see section 5.1.2). We further show that Hebbian descent outperforms Hebb's rule and the covariance rule in general (see section 5.1), has a performance similar to gradient descent in batch or mini-batch learning for several epochs (see section 5.1.5), and most importantly, has much better performance than all the other update rules in continual learning (see section 5.1.2). Moreover, we demonstrate empirically that Hebbian descent converges even if the derivative of the activation function is merely positive and can be used with nonlinearities like the step function (see sections 5.1.5 and 5.1.2). Section 5.1.4 illustrates that only Hebbian descent with centering shows a gradual linear forgetting that does not require an additional forgetting mechanism such as weight decay. In section 5.2, we present experiments with various deep network architectures to show that Hebbian descent has similar performance advantages in comparison to gradient descent in that setting, which ties in with well-known state-of-the-art deep learning best practices. To sum up, with our experiments, we demonstrate an overall beneficial effect for performance also in deep learning by counteracting the potentially negative effect of saturating activation functions solely in the output layer.

2 Gradient Descent in Centered Neural Networks

In this work we consider artificial neural networks in which every neuron is a centered neuron (LeCun et al., 2012; Melchior et al., 2016) given by

$$h_j = \phi(a_j) := \phi\left(\sum_i^N w_{ij}(x_i - \mu_i) + b_j\right), \quad (2.1)$$

with input x_i , offset value μ_i (usually the mean of the input component i), bias b_j , weight w_{ij} , activation function $\phi(\cdot)$, and prethreshold activity a_j , where $i \in 1, \dots, N$ and $j \in 1, \dots, M$. Our discussion covers various activation functions such as linear/identity, sigmoid, step, softmax, and rectifier (Hahnloser et al., 2000; Hahnloser & Seung, 2001), as well as exponential linear units (Clevert et al., 2015), which altogether represent most of the

frequently used activation functions in artificial neural networks. The activity for all neurons $\mathbf{h}$ within one layer can be written compactly in matrix notation as

$$\mathbf{h} = \phi(\mathbf{a}) := \phi(\mathbf{W}^T(\mathbf{x} - \boldsymbol{\mu}) + \mathbf{b}) \quad (2.2)$$

$$= \phi\left(\mathbf{W}^T\mathbf{x} + \underbrace{\mathbf{b} - \mathbf{W}^T\boldsymbol{\mu}}_{\mathbf{b}'}\right) \quad (2.3)$$

$$= \phi\left(\mathbf{W}^T\left(\mathbf{x} - \underbrace{\boldsymbol{\mu} + (\mathbf{W}^T)^{-1}\mathbf{b}}_{\boldsymbol{\mu}'}\right)\right), \quad (2.4)$$

with weight matrix $\mathbf{W}$, element-wise activation function $\phi(\cdot)$, input $\mathbf{x}$, output $\mathbf{h}$, offset $\boldsymbol{\mu}$, bias $\mathbf{b}$, and prethreshold activity $\mathbf{a}$. Note that throughout this work, we consider all vectors to be column vectors by default and therefore row vectors if transposed.

Centering in neural networks refers to the subtraction of the mean activity from each neuron, so that all neurons have zero mean activity on average.¹ It has been shown to be useful for training artificial neural networks (LeCun et al., 2012), in particular for training Boltzmann machines (Montavon & Müller, 2012; Melchior et al., 2016) and autoencoder networks (Melchior et al., 2016), as it makes the network independent of the first-order input statistics, that is, the mean value of each neuron. Centering thus prevents the network from representing mean information through the weights, which allows it to learn only the missing information of a newly represented pattern rather than having to store it entirely in the weights (Melchior, 2021). This is presumably important in continual learning, where we want to store the latest pattern as much as possible while interfering with the stored patterns as little as possible. Since the mean for hidden units is usually not known in advance and changes during training, the offsets can be updated during learning, for example, by an exponentially moving average. When centering was originally proposed by LeCun et al. (2012) and Schraudolph (1998) the authors also recommended normalizing the units' input to have the same variance, which if updated online as well is closely related to batch normalization proposed by Ioffe and Szegedy (2015).

An important property of both centering as well as full input normalization is that neither of them changes the model class; that is, each centered or normalized artificial neural network can be reparameterized to an uncentered or unnormalized neural network and vice versa, and is therefore just a different parameterization of the same model (Melchior, 2021). Notice also that both centering and normalization are independent of the used

¹ Note that the average here can refer to the average activity over the batch, the training data seen so far, or the whole training data.

learning rule, which is usually gradient descent or backpropagation (Kelley, 1960; Rumelhart, Hinton, et al., 1986) or contrastive learning (Rumelhart, McClelland, et al., 1986; Hinton, 2002) but can also be Hebbian learning (Hebb, 1949).

The most frequently used method to train neural networks now is by iterative parameter updates via gradient descent on the loss function. Thereby, each parameter is updated by a small fraction of the partial derivative of the loss function with respect to that parameter. For the output layer of a neural network as given by equation 2.2, the negative partial derivatives with respect to weights and biases are given in matrix notation by

$$\Delta_{GD} \mathbf{W} = -\frac{\eta \partial \mathcal{L}(\mathbf{t}, \mathbf{h})}{\partial \mathbf{W}} = -\eta \left(\frac{\partial \mathcal{L}(\mathbf{t}, \mathbf{h})}{\partial \mathbf{W}^T} \right)^T \quad (2.5)$$

$$= -\eta \left(\frac{\partial \mathcal{L}(\mathbf{t}, \mathbf{h})}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{W}^T} \right)^T \quad (2.6)$$

$$= -\eta \left(\frac{\partial \mathbf{a}}{\partial \mathbf{W}^T} \right)^T \left(\frac{\partial \mathcal{L}(\mathbf{t}, \mathbf{h})}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{a}} \right)^T \quad (2.7)$$

$$\stackrel{(2.2)}{=} -\eta (\mathbf{x} - \boldsymbol{\mu}) (\mathcal{E}(\mathbf{t}, \mathbf{h}) \odot \boldsymbol{\phi}'(\mathbf{a}))^T, \quad (2.8)$$

$$\Delta_{GD} \mathbf{b} = -\frac{\eta \partial \mathcal{L}(\mathbf{t}, \mathbf{h})}{\partial \mathbf{b}} = -\eta \left(\frac{\partial \mathcal{L}(\mathbf{t}, \mathbf{h})}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{a}} \frac{\partial \mathbf{a}}{\partial \mathbf{b}} \right) \quad (2.9)$$

$$= -\eta \left(\frac{\partial \mathcal{L}(\mathbf{t}, \mathbf{h})}{\partial \mathbf{h}} \frac{\partial \mathbf{h}}{\partial \mathbf{a}} I_M \right) \quad (2.10)$$

$$\stackrel{(2.2)}{=} -\eta (\mathcal{E}(\mathbf{t}, \mathbf{h}) \odot \boldsymbol{\phi}'(\mathbf{a})), \quad (2.11)$$

in which η is the learning rate, $\odot$ denotes the Hadamard product (element-wise product), and $\boldsymbol{\phi}'(\mathbf{a})$ denotes the derivative of the activation function applied element-wise to the input vector, that is, $\boldsymbol{\phi}'([a_0, \dots, a_M]) = [\phi'(a_0), \dots, \phi'(a_M)]$. We furthermore use the identity $\mathcal{E}(\mathbf{t}, \mathbf{h}) = \frac{\partial \mathcal{L}(\mathbf{t}, \mathbf{h})}{\partial \mathbf{h}}$ and call it the error signal.²

²The elementwise derivation of the error signal is

$$\begin{aligned} \Delta_{GD} w_{ij} &\stackrel{(2.6)}{=} -\eta \frac{\partial \mathcal{L}_{HD}(t_j, h_j)}{\partial h_j} \frac{\partial h_j}{\partial a_j} \frac{\partial a_j}{\partial w_{ij}} \\ &\stackrel{(2.1)}{=} -\eta \frac{\partial \mathcal{L}_{HD}(t_j, h_j)}{\partial h_j} \frac{\partial \phi(a_j)}{\partial a_j} \frac{\partial (\sum_i^N w_{ij} (x_i - \mu_i) + b_j)}{\partial w_{ij}} \\ &= -\eta (x_i - \mu_i) \mathcal{E}(t_j, h_j) \phi'(a_j), \\ \Delta_{GD} b_{ij} &\stackrel{(2.6, 2.1)}{=} -\eta \frac{\partial \mathcal{L}_{HD}(t_j, h_j)}{\partial h_j} \frac{\partial \phi(a_j)}{\partial a_j} \frac{\partial (\sum_i^N w_{ij} (x_i - \mu_i) + b_j)}{\partial b_j} \\ &= -\eta (x_i - \mu_i) \mathcal{E}(t_j, h_j) \phi'(a_j). \end{aligned}$$

Equations 2.8 and 2.11 clearly show that the derivative of the output layer's activation function plays a key role in the parameter updates. Here a problem can occur when the prethreshold activities $\mathbf{a}$ of the output layer take values that, passed through the derivative of the activation function, lead to zero or near zero results ($\phi'(a_j) \approx 0$). In this case, the corresponding partial derivatives become zero or get close to zero independent of the actual error signal $\mathcal{E}(t_j, h_j)$ (e.g. $h_j - t_j$), as can be seen from equations 2.8 and 2.11. Once this saturated region of the activation function is reached, updating the network parameters significantly can require a large number of update steps. In case of batch or mini-batch learning with an appropriate parameter initialization and a sufficiently small learning rate, this effect, as shown in the experiments, is usually less of an issue since the network is presented with a balanced variety of input-output combinations in every learning step. This reduces the risk of the network to "burn in" wrong outputs far into the saturated regime of ϕ for a majority of the training samples. In continual learning, however, in which we want to store patterns more or less instantaneously or when learning nonstationary input distributions, this problem becomes more severe.

3 Hebbian Descent

The question arises whether there are any mitigation strategies available to prevent gradient descent from getting stuck in input regions where the derivative of the output-layer activation function is near zero but the error signal is still large. As the activation function is usually well selected depending on the problem at hand, the issue we describe can certainly be considered an unwanted side effect. An effective solution to this issue could be to exclude the derivative of the activation function during the gradient-descent update process. This approach may seem ad hoc, but it can be implemented correctly by modifying the loss function, given that the derivative of the output layer's activation function is always positive:

$$\mathcal{L}_{HD}(\mathbf{t}, \mathbf{h}) = \int \left(\frac{\frac{\partial \mathcal{L}(\mathbf{t}, \mathbf{h})}{\partial \mathbf{h}}}{\frac{\partial}{\partial \mathbf{a}} \odot \mathbf{h}} \right) d\mathbf{h} = \int \frac{\mathcal{E}(\mathbf{t}, \mathbf{h})}{\phi'(\mathbf{a})} d\mathbf{h}, \quad (3.1)$$

where fraction bars denote element-wise division (if not derivatives), $\int$ denotes the indefinite integral, and $\mathcal{L}(\mathbf{t}, \mathbf{h})$ represents some standard gradient descent loss function. If we insert $\mathcal{L}_{HD}$ into equations 2.7 and 2.10 the integrals with respect to $\mathbf{h}$ cancel out with the partial derivative with respect to $\mathbf{h}$ and so do the partial derivatives of the activation functions in the numerator and denominator. Consequently, the updates for the output layer's weight matrix $\mathbf{W}$ and bias vector $\mathbf{b}$ become

$$\Delta_{HD} \mathbf{W} = -\eta(\mathbf{x} - \boldsymbol{\mu}) \frac{\partial \mathcal{L}(\mathbf{t}, \mathbf{h})}{\partial \mathbf{h}}^T \quad (3.2)$$

$$= -\eta(\mathbf{x} - \boldsymbol{\mu}) \mathcal{E}(\mathbf{t}, \mathbf{h})^T, \quad (3.3)$$

$$\Delta_{HD} \mathbf{b} = -\eta \frac{\partial \mathcal{L}(\mathbf{t}, \mathbf{h})}{\partial \mathbf{h}} \quad (3.4)$$

$$= -\eta \mathcal{E}(\mathbf{t}, \mathbf{h}), \quad (3.5)$$

with learning rate or step-size parameter η and error signal $\mathcal{E}(\mathbf{t}, \mathbf{h}) = \frac{\partial \mathcal{L}(\mathbf{t}, \mathbf{h})}{\partial \mathbf{h}}$ with corresponding loss function $\mathcal{L}(\mathbf{t}, \mathbf{h})$.

We call the learning rule originating from the loss modification presented in equation 3.1 Hebbian descent, giving credit to Hebbian learning as well as gradient descent, both of which it is strongly connected to. In case of shallow networks using the MSE loss and particular activation functions, this update rule has previously been discussed in the context of generalized linear models (Nelder & Baker, 1972; Kakade et al., 2011; Goel et al., 2018), contrastive Hebbian learning (Rumelhart, McClelland, et al., 1986; Movellan, 1991; Xie & Seung, 2003), and gradient descent or delta rule (Hinton, 1989; Hertz et al., 1997). Moreover, in the special case of binary classification using the MSE loss and a step activation function, Hebbian descent realizes the perceptron update rule proposed by Rosenblatt (1958). If a linear activation function is used instead of the step activation function, Hebbian descent implements the delta rule or Widrow-Hoff rule (Widrow & Hoff, 1960). With this work, we aim to provide a unified view on all of the algorithms we have noted.

When the integral in equation 3.1 exists, it is clear that Hebbian descent naturally inherits the convergence properties of stochastic gradient descent (Robbins & Siegmund, 1985; Saad, 1998), as it effectively is gradient descent with a different loss function. Note that the integral only exists if the output-layer activation function is strictly monotonic, that is, $\phi'(\mathbf{a}) \neq 0$ everywhere. If it is strictly monotonically decreasing, the integral exists, but $\mathcal{L}_{HD}$ leads to updates in the opposite direction of the updates from the original loss $\mathcal{L}$, which is clearly undesirable. This is the reason that Hebbian descent formally requires a strictly monotonically increasing output-layer activation function. However, in practice we found that in most cases, it is easy to modify a positive activation function to become strictly positive and that Hebbian descent usually performs well even for merely positive activation functions (e.g., a rectifier), in which case the integral exists only partially.

It is important to point out that the optimal parameters with respect to $\mathcal{L}_{HD}$ and $\mathcal{L}$ usually differ as the average loss over all data points differs unless not all individual loss terms are zero. However, in practice, neither $\mathcal{L}$ nor $\mathcal{L}_{HD}$ are the metrics we are interested in, such as when using cross-entropy loss for training a classification network, for example.

Although Hebbian descent is commonly associated with training shallow neural networks, it is equally applicable and also beneficial to deep networks, in which case it only concerns the output-layer activation and loss functions. While it is challenging to adjust the loss function to counteract activation function derivatives in deeper layers, modern deep architectures typically use nonsaturating activation functions like the rectified linear unit

(Hahnloser et al., 2000; Glorot et al., 2011) in their hidden-layers, thus mitigating the issue of vanishing gradients caused by near-zero derivatives. Choosing from a wide range of possible hidden layer activation functions is possible as we usually do not require the activation functions in the hidden layers to represent a certain distribution, while for the output layer, we typically do. This leaves the output layer as the one special case where we might have to use a particular saturating activation function if the use case requires it. In these cases, Hebbian descent can prevent the output layer from blocking the error signal and mitigate potential issues throughout the network. Well-established combinations of loss and activation functions for deep networks have been shown to achieve good performance and can be explained by Hebbian descent. For instance, mean squared loss with linear output units or cross-entropy loss with sigmoid output units both cancel out the derivative of the output-layer activation function during parameter update.³ Hebbian descent provides a general explanation of why these combinations tend to work better than others and, moreover, enable us to design improved pairings of loss and activation functions from an error term perspective that treats them as a unity. A detailed derivation showing that the cross-entropy loss with sigmoid output units indeed follows the Hebbian descent paradigm is given in appendix A.

3.1 The Squared Error Loss. In general a useful loss should define a unique optimum in which the network output $\mathbf{h}$ matches the desired output $\mathbf{t}$, in which case the corresponding error term (i.e., the partial derivative with respect to the input) vanishes. As a concrete example, consider a neural network with sigmoid output activation function ϕ that should be trained via gradient descent to minimize the MSE loss $\mathcal{L}(\mathbf{t}, \mathbf{h}) = \frac{1}{2} (\mathbf{h} - \mathbf{t})^T (\mathbf{h} - \mathbf{t})$. For the sake of the example, we deviate from the usual best practice of using the cross-entropy loss in such a case for a moment. We are now facing the situation of a saturating output-layer activation function, which can cause suboptimal convergence speed. However, following the Hebbian descent idea, we can transform the loss function according to equation 3.1, which effectively yields the cross-entropy loss again. Note that for this special case, Hinton (1989) and Hertz et al. (1997) found that the cross-entropy has the advantage of canceling out the partial derivative of the sigmoid or softmax units in the gradient. The corresponding Hebbian descent parameter updates are

$$\Delta_{HD} \mathbf{W} \stackrel{(2,2,3,3)}{=} -\eta (\mathbf{x} - \boldsymbol{\mu}) \left(\phi((\mathbf{x} - \boldsymbol{\mu}) \mathbf{W} + \mathbf{b}) - \mathbf{t} \right)^T \quad (3.6)$$

$$\stackrel{(2,2)}{=} \underbrace{\eta (\mathbf{x} - \boldsymbol{\mu}) \mathbf{t}^T}_{\text{Sup. Hebb}} - \underbrace{\eta (\mathbf{x} - \boldsymbol{\mu}) \mathbf{h}^T}_{\text{Unsp. Hebb}}, \quad (3.7)$$

³ Although this is trivially the case with a linear activation function.

$$\Delta_{HD} \mathbf{b} \stackrel{(2.2,3.5)}{=} -\eta \left(\phi((\mathbf{x} - \boldsymbol{\mu})\mathbf{W} + \mathbf{b}) - \mathbf{t} \right) \quad (3.8)$$

$$\stackrel{(2.2)}{=} \eta \mathbf{t} - \eta \mathbf{h}. \quad (3.9)$$

Thus, with Hebbian descent in the case of the MSE and sigmoid activation, the network learns to produce a desired output $\mathbf{t}$ given input $\mathbf{x}$ by comparing the current output $\mathbf{h}$ with the desired output $\mathbf{t}$. Equation 3.7 shows that the update rule is the difference between a supervised and an unsupervised Hebb-learning step (see section 3.2). This is strongly connected to contrastive learning rules and the contrastive divergence learning paradigm, and we investigate the connection more closely in the supplementary material. While the supervised learning step measures the correlation between input and desired output, the unsupervised step measures the correlation between input and output that is already represented by the network and removes it from the current update step. This is an important property as it allows learning only the missing information and thus complementing the representation that has already been learned by the network.

To better illustrate the impact of switching from gradient descent with the regular squared error loss to Hebbian descent (i.e., using the cross-entropy loss), Figure 1a shows the different speed of convergence for a network with sigmoid units trained on a 2D toy example using gradient descent and Hebbian descent. It is evident that the norm of the gradient shrinks extremely in saturated regimes of the sigmoid in which the derivative of the activation function takes very small values. Figure 1b illustrates that even if the norm of the update rules is normalized, Hebbian descent converges faster since it points almost directly to the global minimum. Note that due to different optimization dynamics, Hebbian descent and gradient descent arrive at the same optimum only if both achieve exactly zero loss.

3.2 Hebbian Descent from the Perspective of Hebbian Learning. Hebbian learning (Hebb, 1949) is one of the oldest and best-known learning rules for neural network training. It plays a central role in research history and is still relevant today. Although section 3.1 briefly touches on the connection between Hebbian descent and Hebbian learning, we aim to provide a more detailed and formal explanation of this connection.

As shown in equations 3.7 and 3.9 in the case of the squared error loss, Hebbian descent can be considered as the difference between two Hebbian learning terms. By removing the second term in equation 3.7 and fixing the bias term to zero without updating it, we get the centered supervised Hebb rule, which is given by

$$\Delta_H \mathbf{W} = \eta (\mathbf{x} - \boldsymbol{\mu}) \mathbf{t}^T, \quad (3.10)$$

where μ denotes the average over the data. Hebb learning simply adds up the second cross-moments between input and output patterns and

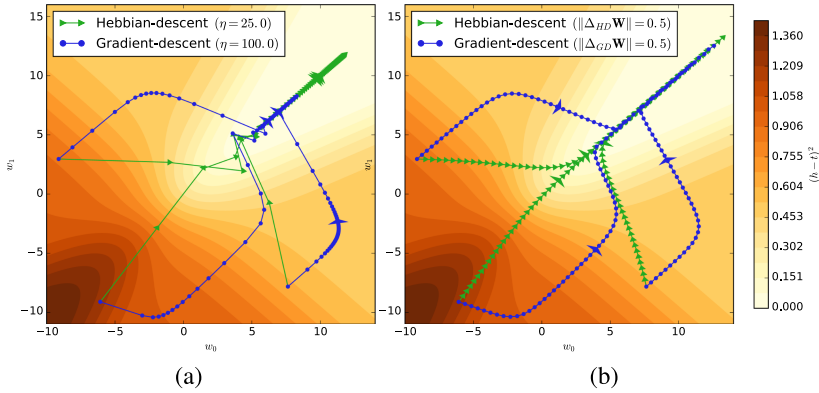


Figure 1: Comparison of gradient descent and Hebbian descent. Both panels show the loss landscape of a 2D toy example and optimization paths for networks trained for 50 full-batch updates (equivalent results are obtained for continual learning) using a squared error loss with either Hebbian descent or gradient descent. Notice that in this setting, Hebbian descent is equivalent to gradient descent with cross-entropy loss (see equation 3.1). The networks have two input units and a single sigmoid output unit, and they are trained on the data set input: $\{(1, 0), (0, 1), (1, 1)\} \rightarrow \text{target} : \{(0), (0), (1)\}$ using a squared-error loss. The wider arrows indicate the optimization step after 25 update steps. (a) The optimization paths for the two methods in which the learning rates were chosen by hand such that both converged without much oscillation. Although Hebbian descent has a smaller learning rate, it converges faster than gradient descent, as can be seen by comparing the wider arrows. (b) The same experiments were performed in which all updates were normalized to have the same norm, illustrating that the direction of Hebbian descent leads to a faster convergence than gradient descent.

therefore does not focus on learning only missing information such as Hebbian descent does. As shown in our experiments, this limits the model in associating arbitrary patterns with each other, which is in particular problematic if the patterns are correlated, as has been analyzed for Hopfield networks by Löwe et al. (1998). Furthermore, in contrast to Hebbian descent and gradient descent, the Hebb rule is not a convergent learning rule since the weights continue to grow with each update step. While this is less of an issue for saturating activation functions such as the sigmoid, in case of nonsaturating activation functions such as identity or rectifier, one needs a mechanism that restricts the weights so that the network output stays in a reasonable range. Usually the weights are rescaled by the number of performed update steps, which has the disadvantage that we need to keep track of the number of updates, or a weight decay term (Bienenstock et al., 1982) is added, which has the disadvantage of introducing an additional hyperparameter that needs to be chosen in advance.

A problem of the original Hebb rule (see equation 3.10 with $\mu = 0$) is that it cannot learn negative or inhibitory weights when the activities are merely positive. In order to compensate for this limitation, Sejnowski and Tesauro (1989) have proposed the covariance rule, which is given for the weights of a single-layer neural network by

$$\Delta_C \mathbf{W} = \eta (\mathbf{x} - \langle \mathbf{x} \rangle) (\mathbf{t} - \langle \mathbf{t} \rangle)^T, \quad (3.11)$$

in which $\langle \cdot \rangle$ is used to denote the average over the data. The covariance rule, as the name suggests, models the covariance between input and output activities, whereas the uncentered Hebb rule models the corresponding second cross moment. For positive input data, the covariances can be positive or negative while the second moments are always positive, so that the covariance rule can learn inhibitory weights while the uncentered Hebb rule cannot. If centering is used, we show in the supplementary material that the centered Hebb rule becomes equivalent to the covariance rule.

Note, however, that, whether centered or not, the covariance rule is still a divergent learning rule and is unable to consider missing information only in the parameter updates, which limits its effectiveness of learning correlated or similar pattern pairs.

3.3 Hebbian Descent and Generalized Linear Models. Generalized linear models (GLMs) are a popular tool in statistics and subsume a variety of simpler models such as ordinary linear or logistic regression. Here we show that in the case of the squared error loss and an invertible and integrable activation function, Hebbian descent actually optimizes a GLM in canonical form (Nelder & Baker, 1972). We first give a brief introduction to GLMs and then show that the gradient of GLMs is equivalent to the Hebbian descent update, which can also be concluded from the derivations given by Nelder & Baker (1972) and McCullagh & Nelder (1989). (For a detailed introduction to GLMs, see McCullagh & Nelder, 1989.)

GLMs model the probability distribution $p(\mathbf{t}|\mathbf{x})$ of output $\mathbf{t}$ given input $\mathbf{x}$. As a generalization of linear least squares regression, which assumes gaussian distributed output variables, GLMs allow the output variables to follow any distribution that is in the natural exponential family. A distribution over output variables $\mathbf{t}$ that is parameterized through $\boldsymbol{\theta}$ (which depends on $\mathbf{x}$; see below) belongs to the natural exponential family if it can be written in the following form,

$$p(\mathbf{t}|\boldsymbol{\theta}) = b(\mathbf{t}) \exp(\boldsymbol{\theta}^T \mathbf{t} - A(\boldsymbol{\theta})), \quad (3.12)$$

$$= \frac{1}{\exp(A(\boldsymbol{\theta}))} b(\mathbf{t}) \exp(\boldsymbol{\theta}^T \mathbf{t}), \quad (3.13)$$

in which $\boldsymbol{\theta}$ is usually known as the natural parameter, $A(\boldsymbol{\theta})$ is the log-partition function, and $b(\mathbf{t})$ is a known function. Equation 3.13 illustrates

more obviously why $A(\theta)$ is named a log-partition function, as in order to normalize the distribution ($\int p(\mathbf{t}|\theta) d\mathbf{t} = 1$), $A(\theta)$ has to be

$$A(\theta) = \log \left(\int b(\mathbf{t}) \exp(\theta^T \mathbf{t}) d\mathbf{t} \right). \quad (3.14)$$

Notice that in the literature for GLMs, a dispersion parameter is often given explicitly, which we have dropped for readability as it can also be modeled implicitly with the given parameterization.

A particular property of distributions of the natural exponential family is that the expectation value of the output variable is known to be

$$\mathbf{E}[\mathbf{t}] = \nabla A(\theta), \quad (3.15)$$

in which $\mathbf{E}[\mathbf{t}]$ denotes the vector containing expectation values of the output variables: $\mathbf{E}[\mathbf{t}] = [E[t_0], \dots, E[t_M]]$. Thus, $\mathbf{E}[\mathbf{t}]$ and θ must be related, which is generally denoted by

$$\theta = \psi(\mathbf{E}[\mathbf{t}]) \quad (\text{implying } \nabla A(\theta) \text{ to be invertible}), \quad (3.16)$$

in which $\psi(\cdot)$ is known as the link function, which is an invertible mapping that relates the model mean to the natural parameter. In order to model $\mathbf{E}[\mathbf{t}]$ based on the input $\mathbf{x}$, one decides for a linear predictor $\mathbf{a}$ (i.e., a linear combination of the input values and some parameters) that is passed through an invertible, integrable, and possibly nonlinear function $\phi(\mathbf{a}) = \mathbf{E}[\mathbf{t}] = \nabla A(\theta)$. Such a mapping is provided by a centered single-layer network (see equation 2.2) when the activation function is chosen to be invertible and integrable, in which case the natural parameter becomes

$$\theta \stackrel{(2.2, 3.16)}{=} \psi(\phi(\mathbf{a})) = \psi(\phi(\mathbf{W}^T (\mathbf{x} - \mu) + \mathbf{b})). \quad (3.17)$$

We can now use maximum likelihood estimation to optimize the parameters of the model. We consider the gradient of the model's LL with regard to $\mathbf{W}$ and $\mathbf{b}$, which is given by

$$\eta \frac{\partial \log p(\mathbf{t}|\mathbf{x}, \mathbf{W}, \mathbf{b})}{\partial \mathbf{W}} \stackrel{(3.12)}{=} \eta \frac{\partial}{\partial \mathbf{W}} \log(b(\mathbf{t})) + \eta \frac{\partial}{\partial \mathbf{W}} (\theta^T \mathbf{t} - A(\theta)) \quad (3.18)$$

$$\stackrel{(3.17)}{=} \eta (\mathbf{x} - \mu) \left((\mathbf{t} - \nabla A(\theta)) \odot \psi'(\phi(\mathbf{a})) \odot \phi'(\mathbf{a}) \right)^T \quad (3.19)$$

$$\stackrel{(\phi(\mathbf{a}) = \nabla A(\theta))}{=} \eta (\mathbf{x} - \mu) \left((\mathbf{t} - \phi(\mathbf{a})) \odot \psi'(\phi(\mathbf{a})) \odot \phi'(\mathbf{a}) \right)^T, \quad (3.20)$$

$$\eta \frac{\partial \log p(\mathbf{t}|\mathbf{x}, \mathbf{W}, \mathbf{b})}{\partial \mathbf{b}} \stackrel{(3.12, 3.20)}{=} \eta \left((\mathbf{t} - \phi(\mathbf{a})) \odot \psi'(\phi(\mathbf{a})) \odot \phi'(\mathbf{a}) \right). \quad (3.21)$$

In order to be able to calculate the gradient numerically, we have to decide for an activation function, where we are free to choose any function as long as it is invertible and integrable. However, there is a striking choice: to choose $\phi(\cdot) = \psi^{-1}(\cdot)$ (Nelder & Baker, 1972), in which case the model is said to be in canonical form since the natural parameter simplifies to $\theta = \mathbf{a}$ so that the gradient becomes

$$\eta \frac{\partial \log p(\mathbf{t}|\mathbf{x}, \mathbf{W}, \mathbf{b})}{\partial \mathbf{W}} \stackrel{(3.12, \theta=\mathbf{a})}{=} \eta \frac{\partial}{\partial \mathbf{W}} \log(b(\mathbf{t})) + \eta \frac{\partial}{\partial \mathbf{W}} (\mathbf{a}^T \mathbf{t} - A(\mathbf{a})) \quad (3.22)$$

$$= \eta (\mathbf{x} - \boldsymbol{\mu})(\mathbf{t} - \nabla A(\mathbf{a}))^T \quad (3.23)$$

$$\stackrel{\phi(\mathbf{a})=\nabla A(\theta)}{=} \eta (\mathbf{x} - \boldsymbol{\mu})(\mathbf{t} - \phi(\mathbf{a}))^T, \quad (3.24)$$

$$\eta \frac{\partial \log p(\mathbf{t}|\mathbf{x}, \mathbf{W}, \mathbf{b})}{\partial \mathbf{b}} \stackrel{(3.21, 3.24)}{=} \eta (\mathbf{t} - \phi(\mathbf{a})). \quad (3.25)$$

This update rule is equivalent to the Hebbian descent update for a squared error loss as given by equations 3.6 and 3.8 showing that the Hebbian descent update with squared error loss and an invertible and integrable activation function actually optimizes the LL of a GLM in canonical form.

Note that Kakade et al. (2011) have proposed an algorithm named L-Isotron that uses the gradient of canonical GLMs (see equations 3.24 and 3.25), but additionally allows learning the nonlinearity instead of choosing it by hand. The authors have also provided convergence bounds for the gradient of GLMs in canonical form. Goel et al. (2018) have transferred the algorithm to single-layer convolutional neural networks.

4 Methods

In this section, we describe the benchmark data sets and the experimental setup. For this work, we used the machine learning library PyDeep, which allows reproducing the described results and provides examples.⁴ We chose a mix of community standard plus one random data set, which can all be considered simple by today's standards. This was done to maintain focus on the comparison aspect of the different methods, reduce experiment run times, and increase the chances of readers being already familiar with the used data sets. To bring each network architecture and learning rule to its full potential, we performed a grid search over the most influential hyperparameters per experiment (see sections 4.2 and 4.3). Again to reduce experiment run times, we chose the particular set of hyperparameters we

⁴Machine learning library PyDeep: <https://pydeep.readthedocs.io/>, Example for Hebbian descent: https://github.com/MelJan/PyDeep/blob/master/examples/Example_Hebbian_descent.py.

would perform a grid search over based on initial experiments and domain knowledge.

4.1 Benchmark Data Sets. We consider four real-world data sets from various domains as well as binary and normal distributed random patterns in our experiments. For all classification data sets, the class labels are presented as one-hot vectors to the networks.

The MNIST (LeCun et al., 2012) data set consists of 70,000 gray-scale images of handwritten digits divided into training and test sets of 60,000 and 10,000 patterns, respectively. The images have a size of 28×28 pixels, in which all pixel values are normalized to the range $[0, 1]$. The data set is not binary, but the values tend to be close to zero or one. Each pattern is assigned to one out of 10 classes representing the digits 0 to 9.

The CONNECT (Larochelle et al., 2010) data set consists of 67,587 game-state patterns from the game Connect-4. The data set is divided into training, validation, and test sets with 16,000, 4000, and 47,557 patterns, respectively. The binary patterns are 126-dimensional, and each pattern is assigned to one out of three classes representing the game results: win, lose, or draw.

The ADULT (Larochelle et al., 2010) data set consists of 32,561 binary patterns of census data to predict whether a person's income exceeds \$50,000 per year. The data set is divided into training, validation, and test sets with 5000, 1414, and 26,147 patterns, respectively. The binary patterns are 123-dimensional, and each pattern is assigned to one out of two classes representing whether the income level was exceeded or not.

The CIFAR (Krizhevsky, 2009) data set consists of 60,000 color images of various objects divided into training, validation, and test sets with 40,000, 10,000, and 10,000 patterns, respectively. The images have a size of 32×32 pixels that are converted to gray scale and rescaled to lie in a range of $[0, 1]$, so that the data set has a nonzero mean and can be represented by most of the activation functions. Each pattern is assigned to one out of 10 classes representing trucks, cats, or dogs, for example.

The RAND and RANDN data sets serve as baseline data sets, each consisting of random patterns with a size of 200 pixels. The pixels in the RAND data set take the value one with a probability of 0.5 and zero otherwise. The pixels in the RANDN data set are drawn from a gaussian distribution with zero mean and unit variance. The data set is rescaled to lie in a range of $[0, 1]$, so that it can be represented by most of the activation functions. The resulting data set has a mean of 0.5 and standard deviation of 0.1. These data sets do not have label information.

4.2 Network Structure and Learning Setup for Single-Layer Network Experiments. In our main experiments, we consider single-layer networks with various activation functions such as linear, sigmoid, step, softmax, rectifier, and exponential linear. Unless stated otherwise, in the experiments we

used MSE error loss without weight decay regularization. Here we focus on single-layer networks to make assessing the comparison between Hebbian descent and some of the other learning rules feasible. The bias values were initialized to zero, and according to Glorot and Bengio (2010), we initialized the weights to $w_{ij} \sim U\left(-\frac{\sqrt{6}}{\sqrt{N+M}}, +\frac{\sqrt{6}}{\sqrt{N+M}}\right)$, in which N is the number of input units, M is the number of output units, and $U(a, b)$ is the uniform distribution in the interval $[a, b]$. Each experiment was repeated 10 times, in which the initial weight matrices were the same among the methods but different in each trial. The default batch size was one in case of continual learning and 100 in case of mini-batch learning, and when training involved several sweeps through the data, the models were trained for 100 epochs. Depending on the data set and activation function, the optimal learning rate varied a lot, so we performed a grid search over 35 different learning rates ranging from 0.00002 to 100.⁵ When weight decay was used, we additionally performed a grid search over 20 different weight decay values ranging from 0 to 2 leading to a total search space of $20 \times 35 = 700$ hyper-parameter combinations.⁶ When centering was used and if not mentioned otherwise, the input offsets were fixed to the corresponding data mean, and the hidden offsets were initialized to $\lambda_j(t = 0) = 0.5$ and updated with an exponential moving average of $\lambda_j(t + 1) = 0.99\lambda_j(t) + 0.01h_j(t)$. For a fair comparison of the methods, we had to fix input offsets to the data mean because changing offsets during training requires a bias parameter for the reparameterization that is not available when using Hebb's rule and the covariance rule. However, slowly updated input offsets converge to the data mean, leading to a very similar performance as when initially fixing them to the data mean. This has been shown for mini-batch learning in restricted Boltzmann machines by Melchior et al. (2016) and is shown for continual heteroassociation in the following. Without centering, the offsets were all fixed to zero.

4.3 Network Structure and Learning Setup for Multilayer Network Experiments. We provide classification experiments with deep networks on the MNIST and CIFAR data sets as well as regression experiments using a denoising autoencoder on CIFAR to illustrate the efficacy of Hebbian descent in this setting. Network parameter initialization and search for the optimal learning rate were done in the same way as described above for the single-layer network experiments. Each experiment was repeated five times with the same initial weight matrices among the methods but different in

⁵We used the following learning rates in this work: 0.00002, 0.00004, 0.00006, 0.00008, 0.0001, 0.0002, 0.0004, 0.0006, 0.0008, 0.001, 0.002, 0.004, 0.006, 0.008, 0.01, 0.02, 0.04, 0.06, 0.08, 0.1, 0.2, 0.4, 0.6, 0.8, 1, 2, 4, 6, 8, 10, 20, 40, 60, 80, 100.

⁶We used the following weight decay values in this work: 0.0, 0.0001, 0.0005, 0.001, 0.002, 0.004, 0.006, 0.008, 0.01, 0.02, 0.04, 0.06, 0.08, 0.1, 0.2, 0.4, 0.6, 0.8, 1.0, 2.0.

each trial. We test the combination of softmax output-layer activation with MSE or CE loss, representing the gradient descent and a Hebbian descent setting, respectively. Additionally, in the classification experiments, we pair the MSE loss with a linear output-layer activation to test how a nonstandard Hebbian descent error term performs in comparison with the best practice of softmax plus CE loss. We use four different network architectures in total, whereas the first three are used in the classification experiments and the last one in the regression experiments:

- A two-layer fully connected network with 200 neurons in both layers and a ReLU nonlinearity between them.
- A two-layer convolutional network with 32 and 64 channels, a stride of 1, and a final fully connected layer with 128 neurons. All hidden layers have ReLU nonlinearities between them.
- A ResNet18 instance, which is an 18-layer deep convolutional network with residual connections and ReLU nonlinearities (He et al., 2016).
- A six-layer autoencoder with 12, 24, 48, 24, 12, 3 channels per layer, a stride of 1, and ReLU nonlinearities between the hidden layers.

4.4 Performance Measurement. As the different methods and activation functions effectively optimize different loss functions, we decided for a coherent performance measure that represents the performed task best and favors neither one method nor the other. For the classification experiments, we therefore evaluated the average misclassification rate, while for the other experiments, we used the mean absolute error (MAE) as an intuitive performance measure. For all supervised experiments (see section 5.2), the reported performance was measured on a held-out test set. For the heteroassociation experiments (see section 5.1), the concept of a test set is not applicable as the task is to explicitly associate a given pattern with one specific other pattern. For the experiments with binary output patterns consisting of zeros and ones, it is clear that a mean absolute error of 0.1, for example, means that on average, each neuron deviates 10% and that an error above 0.5 is worse than for random output (chance level). Alternatively we could have used the mean squared error that would have been in favor for gradient descent, while the log-likelihood would have been in favor for Hebbian descent, thus both not allowing for an unbiased performance measure. Furthermore, both overestimate outliers and underestimate small deviations, thus leading to a less interpretable performance measure. Another choice that is often used in neuroscience is the Pearson correlation between target and output pattern. However, it is scale invariant and can lead to a wrong impression of the network's performance. For some experiments, we also evaluated these other performance measurements and found qualitatively the same results: if a method performed significantly better in terms of the MAE, it was also the best with regard to the other measures.

5 Results

We compare the performance of Hebbian descent, gradient descent, Hebb's rule, and the covariance rule for centered and uncentered single-layer networks. We first focus on continual heteroassociation, associating N input patterns with N output patterns one after the other, but we also present multi-epoch experiments to explore how much the individual learning rules profit from seeing training examples multiple times. In a separate set of experiments, we compare the classification and regression performance of Hebbian descent and gradient descent using several deep network architectures.

5.1 Heteroassociative Learning. In this section we investigate the performance of Hebb's rule, the covariance rule, gradient descent, and Hebbian descent in heteroassociative learning with a focus on continual learning. To maintain an emphasis on the continual learning setting, every data point is generally presented only once to each algorithm.

5.1.1 Heteroassociative Continual Learning without Centering. In a first set of experiments, we analyze how well 100 patterns of one data set can be associated with 100 patterns of another data set one pair at a time. Figure 2a shows the performance of uncentered networks with sigmoid units when the four different methods have been used to associate 100 binary random patterns (RAND) with 100 binary random patterns (RAND). Unless stated otherwise, the optimal learning rate was generally determined for each method individually such that the average performance over the last 20 patterns is best. For select experiments, we added a setting where the learning rate was tuned to achieve the best possible performance for only the last pattern. Note that during performing the learning rate grid search, we found that generally Hebbian descent was more robust with regard to variations of the learning rate, which actually puts the remaining methods at a slight advantage since they profited from the learning rate fine tuning to a higher degree than Hebbian descent.

For Hebb's rule, the error is close to 0.5, which means that the network has not learned to associate the patterns at all. It is the same performance as baseline, which is the error between the output patterns and their mean value. This corresponds to the performance of a network that independent of the input always returns the mean output pattern and thus represents the most trivial solution. The bad performance of Hebb's rule is a direct consequence of not being able to learn negative or zero correlations and can be explained as follows. Equation 3.10 indicates that in the uncentered case and for binary patterns, each weight w_{ij} is updated by a value of η when the corresponding input and target value is one, or by zero otherwise. Since the patterns are drawn uniformly at random, the weights will thus increase with a probability of 0.25 or stay the same otherwise. Consequently,

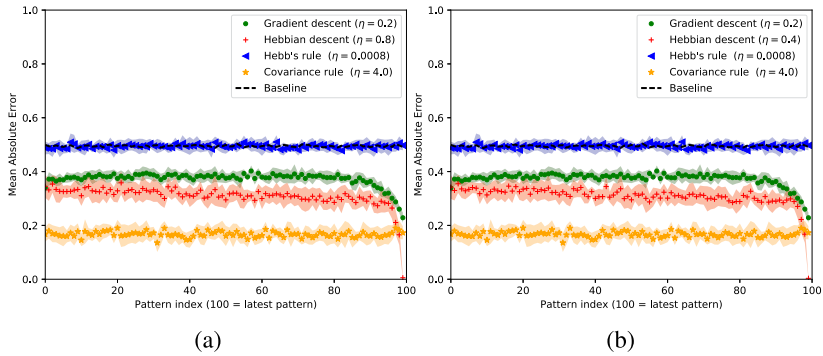


Figure 2: Continual learning performance of the four different update rules with sigmoid units and without centering, when 100 binary random patterns (RAND) are associated with another 100 binary random patterns (RAND) one pattern pair at a time. The mean absolute error and the corresponding standard deviation over 10 trials is plotted for each pattern separately. The learning rate η was chosen for each method individually such that (a) the performance over the final 20 patterns is best and for comparison (b) the performance for the final pattern is best. The baseline (overlaid with the curve for Hebb's rule) represents the performance of a network that independent of the input always returns the mean of the output patterns.

all neurons will sooner or later and independent of the input produce a constant output of one resulting in an error of 0.5.

The covariance rule solves this problem by allowing negative weight changes leading to a much better performance, which is even better than that of Hebbian descent and gradient descent. Interestingly, in the case of Hebb's rule and the covariance rule, early and late patterns are learned equally well, whereas for gradient descent and Hebbian descent, more recent patterns are represented better than older ones. While this enables the latter two approaches to learn continually from new data, both show catastrophic forgetting for older patterns (see Figure 2). Note that only Hebbian descent allows storing the latest pattern almost perfectly, independent of the number of the stored patterns. For the covariance rule, however, the performance of all patterns decreases with an increasing number of learned patterns.

To show that the observed performance differences do not depend on the number of patterns on which the learning rate is optimized, we performed two additional experiments. In one, we selected for each learning rule the optimal learning rate such that the performance of only the last pattern (index 100) is best; the results are shown in Figure 2b. In a second one, the optimal learning rate per learning rule was chosen such that the performance over all 100 patterns is best (results not shown here). For all scenarios, the

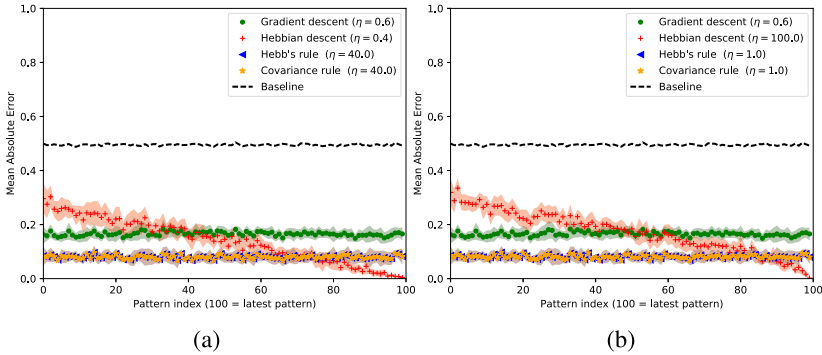


Figure 3: Continual learning performance of the four different update rules with sigmoid units and centering when 100 binary random patterns (RAND) are associated with another 100 binary random patterns (RAND) one pattern pair at a time. The MAE and the corresponding standard deviation over 10 trials is plotted for each pattern separately. The learning rate η was chosen for each method individually such that (a) the performance over the final 20 patterns is best and (b) the performance for the final pattern is best. The baseline represents the performance of a network that independently of the input always returns the mean of the output patterns. See Figure 2 for the same experiment but without centering.

performance curves of the learning rules are very similar. Since the optimal learning rates in the experiments are nearly the same, a larger learning rate does not allow the methods to represent only the latest pattern better.

5.1.2 Heteroassociative Continual Learning with Centering. To show the importance of centering and its ability to prevent catastrophic interference in continual learning when combined with Hebbian descent, we performed the same experiments as before but with centered networks. The results are shown in Figure 3, illustrating that all methods profit significantly from centering and that the covariance rule and Hebb's rule become equivalent in case of centering as shown analytically in equation S6. We refer to Melchior (2021) for a similar performance comparison where the data are not centered to the mean but to offsets learned during training. In the mean-centered setting presented here, all methods except for Hebbian descent have a homogeneous error distribution over the single patterns, which also does not change when the learning rate is selected such that the last pattern is represented best, as shown in Figure 3b. Interestingly, only Hebbian descent allows for a linear slope of forgetting. It is still able to learn from new patterns but at the same time does not suffer from catastrophic interference anymore. While compared to noncentered networks, the performance improved for Hebbian learning, and the covariance rule, the performance of all patterns, will decrease with an increasing number of learned patterns.

We see the reason that Hebbian descent shows this continual learning behavior in two factors. First, centering the data helps the network to disentangle the weights and biases, which effectively avoids large, disruptive weight updates due to the inherent bias of the presented samples (Melchior, 2021). This enables Hebbian descent as well as gradient descent to make more targeted updates with respect to the currently presented data point without compromising what has been learned before. If centering is not applied, we can see catastrophic forgetting as in Figure 2. Second, storing a pattern within one update requires a rather large step size, which easily pushes the units into the saturated regions of their activation functions. While Hebbian descent can potentially undo these steps linearly with respect to the error term (see equations 3.2 and 3.4), gradient descent would need several steps as the update is significantly scaled down by the derivative of the activation function (see equations 2.8 and 2.11), hindering the network from storing the information within one step. Thus, centering is required to enforce disentangling representation of weights and biases, while only Hebbian descent is able to unlearn or forget information that is stored in rather saturated regions of the activation function.

Since natural data are usually not uncorrelated, we performed the same experiments as before but with correlated real-world data sets. In particular, we associated 100 binary random patterns (RAND) with 100 patterns of the ADULT data set. The results for centered networks are shown in Figure 4a, where Hebb's rule and the covariance rule perform significantly worse than gradient descent, Hebbian descent, and also as baseline. Again, Hebbian descent has a linear slope of forgetting while all the other methods have a homogeneous error distribution. The plot for associating a set of patterns of the ADULT data set with another set of patterns of the ADULT data set is qualitatively similar and is thus not shown. Figure 4b shows the reverse experiment in which 100 patterns of the ADULT data set in the input are associated with 100 binary random patterns (RAND) in the output. The error for all methods is rather large, which arises from a rather high pairwise correlation of the patterns in the ADULT data set. Associating two very similar patterns with two completely random output patterns is a rather difficult task in continual learning as the chance of overwriting associations is extremely high. Again, all methods have roughly the same error across all individual patterns except for Hebbian descent, which allows storing at least the most recent patterns with high accuracy. This comes at the cost of the older pattern's accuracy, leading to gradual forgetting. The optimal learning rate in both experiments was chosen such that the performance is best at the last 20 patterns, but consistent with the previous experiments, the results are almost the same when choosing it to be best for only the last or all patterns. Furthermore, without centering, all methods perform significantly worse (data not shown).

To confirm empirically that Hebbian descent is generally better on recent patterns, we performed several experiments with various data sets and

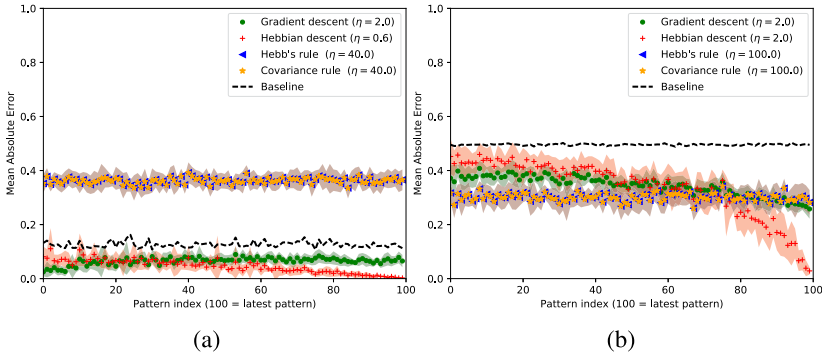


Figure 4: Continual learning performance of the four different update rules with centering and sigmoid units when (a) 100 binary random patterns (RAND) are associated with 100 patterns of the ADULT data set and (b) 100 patterns of the ADULT data set are associated with 100 binary random patterns (RAND). The mean absolute error and the corresponding standard deviation over 10 trials is plotted for each pattern separately. The learning rate η was chosen for each method individually such that the performance over the final 20 patterns was best. The baseline represents the performance of a network that independent of the input always returns the mean of the output patterns.

activation functions. For detailed results, we refer to the supplementary material (see section F, Table S4, as well as Tables S9 and S10). Across all combinations of data sets and activation functions, Hebbian descent and gradient descent perform significantly better than Hebb's rule, and the covariance rule on the last 20 patterns, and the latter two are both not even better than baseline performance. Furthermore, Hebbian descent performs significantly better than gradient descent in most cases, which becomes more significant as the activation function becomes more nonlinear. As an extreme case, we used the step function, which guarantees a binary output of the network but is incompatible with gradient descent as the gradient is constant zero for such networks. Hebbian descent, however, can deal with this type of activation functions and reaches good performance. The linear networks, for which gradient descent and Hebbian descent are equivalent, always perform worse than or similar to corresponding non-linear networks. A nonlinearity should thus be applied, and the sigmoid function has the best performance among the different activation functions even when the output data come from a continuous domain such as ADULT $\rightarrow$ RANDN or CIFAR $\rightarrow$ RANDN, for example (see supplementary material section F, Table S10). To emphasize the advantage Hebbian descent holds over gradient descent in our experiments more clearly, we generated a scatter plot comparing their performance over the various data sets and output-layer activation functions shown in Figure 5. (For more detailed

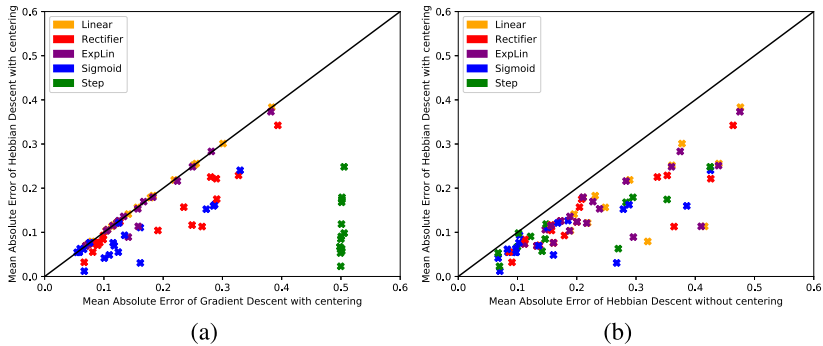


Figure 5: Comparison of continual heteroassociation of (a) Hebbian descent versus gradient descent with centering and (b) Hebbian descent with and without centering. Each cross represents the MAE of the last 20 patterns averaged over 10 trials per experiment.

results, see supplementary material, Tables S4, S9, and S10.) All points lie either roughly on the diagonal or clearly below it, showing that if there is a significant difference between the two methods, Hebbian descent performs significantly better than gradient descent.

For comparison, we performed the same experiments as before but without centering, shown in Figure 5b, which clearly supports our statement that centering is valuable for all methods. Again, more results can be found in the supplementary material: section F, in Tables S5, S11, and S12. Without centering, Hebbian descent loses its ability to store recent patterns significantly better than older ones, but in most cases, it still performs significantly better than the other methods. We also emphasize the superiority of centered over uncentered networks by plotting the results for the centered versus uncentered Hebbian descent experiments in a scatter plot shown in Figure 5b. Detailed experimental results can be found in the supplementary material in Tables S4, S5, S9, S10, S11, and S12. All points lie clearly below the diagonal, showing that centering is always beneficial in our experimental setting.

5.1.3 On the Advantage of Saturating Activation Functions in Continual Learning. The advantage of activation functions with restricted output values is that their values cannot overshoot, meaning that even extreme weight changes will lead to reasonable output values, which seems to be crucial for continual learning. This can be seen from the sensitivity of the network with respect to the learning rate. Figure 6 shows the performance of the last 20 patterns for (a) linear networks and (b) networks with sigmoid activation functions that are trained to associate 100 binary random patterns (RAND) with another 100 binary random patterns (RAND) using different learning

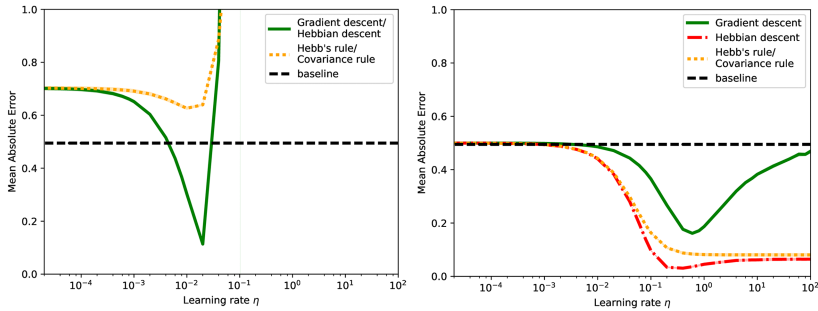


Figure 6: Continual learning performance of the four different update rules as a function of the learning rate. The centered networks have been optimized to associate 100 binary random patterns (RAND) with another 100 binary random patterns (RAND) with (a) linear and (b) sigmoid activation function. The curves represent the average MAE for the final 20 patterns for the corresponding learning rate averaged over 10 trials. The standard deviation is also shown, but the values are too small to be visible without zooming in. The baseline represents the performance of a network that independent of the input always returns the mean of the output patterns.

rates. In the linear case, it is crucial for all methods to choose the right learning rate, as can be seen from the very sharp optimum. If the learning rate is chosen too big, the error increases exponentially. Also notice that Hebb's rule or covariance rule does not even get close to baseline performance. When using a sigmoid activation function, however, all methods perform significantly better than baseline. While gradient descent still has a rather sharp optimum, the performance of the other methods does not change significantly for a learning rate above a certain threshold. This is a useful property as one can simply select a large learning rate, instead of performing a grid search, to achieve performance close to the optimum. Qualitatively, the same picture can be seen for the other data sets used in the experiments.

5.1.4 Heteroassociative Online Learning with Weight Decay. The ability of the network to forget patterns becomes more important as more and more patterns are stored in the network. To illustrate this effect, we trained centered networks to associate 1000 instead of 100 patterns, which is way beyond the capacity of the network. The performance for the last 100 patterns when associating 1000 binary random patterns with another 1000 binary random patterns is shown in Figure 7a and when associating 1000 binary random patterns with 1000 patterns of the ADULT data set is shown in Figure 7b.

The errors for Hebb's rule, the covariance rule, and gradient descent simply increase for all patterns compared to when only 100 patterns have been

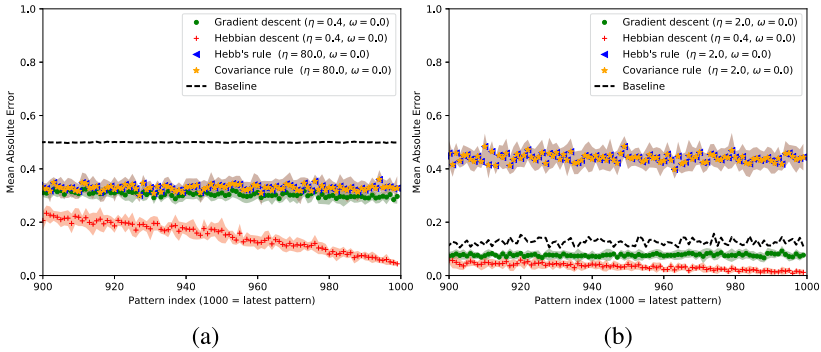


Figure 7: Continual learning performance for the last 100 patterns for the four different update rules with centering and with sigmoid units, when (a) 1000 binary random patterns (RAND) are associated with another 1000 binary random patterns (RAND), and (b) 1000 binary random patterns (RAND) are associated with 1000 patterns of the ADULT data set. The MAE and the corresponding standard deviation over 10 trials are plotted for each pattern separately. The learning rate η was chosen for each method individually such that the performance over the final 20 patterns is best, but optimizing on all or only the final pattern leads to very similar results. The baseline represents the performance of a network that independent of the input always returns the mean of the output patterns.

stored. This can be seen by comparing the results in Figure 7a with Figures 3a and 7b with Figure 4a, respectively. Hebbian descent in contrast still shows a power-law forgetting curve in which more recent patterns are represented better than older ones. Furthermore, only Hebbian descent allows having roughly the same performance on the latest patterns independent of the number of patterns that have been stored previously.

While Hebbian descent controls the amount of forgetting automatically, the other methods need an explicit forgetting mechanism, usually implemented through weight decay. It removes a certain proportion of the current weight matrix in each update step and therefore introduces an additional hyperparameter that controls the speed of forgetting. To investigate the effect of weight decay on the four different methods, we again trained networks to associate 1000 binary random patterns (RAND) with another 1000 binary random patterns (RAND). A simultaneous grid search over learning rate and weight decay was performed such that, just as in the previous experiments, the performance on the final 20 patterns is best.⁷ The results

⁷Note that a different number of patterns (e.g., 50 or 100 patterns) can be chosen for selecting the optimal hyperparameters, which, however, does not alter the conclusion of our analysis.

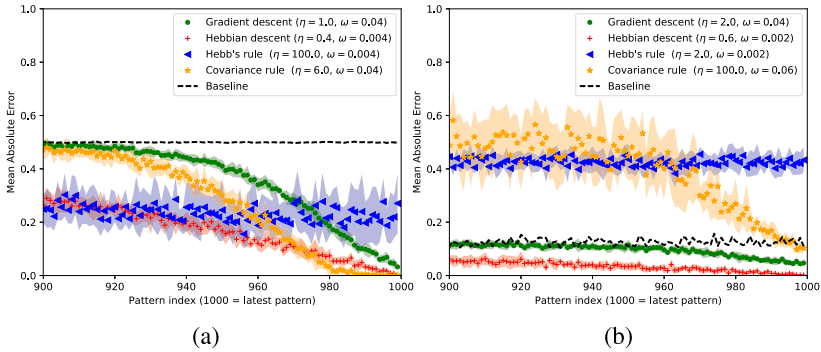


Figure 8: Continual learning performance for the final 100 patterns for the four different update rules with weight decay, centering, and sigmoid units, when (a) 1000 binary random patterns (RAND) are associated with 1000 binary random patterns (RAND) and (b) 1000 binary random patterns (RAND) are associated with 1000 patterns of the ADULT data set. The MAE and the corresponding standard deviation over 10 trials are plotted for each pattern separately. The learning rate η and weight decay ω were chosen for each method individually via grid search such that the performance over the final 20 patterns was best. The baseline represents the performance of a network that independent of the input always returns the mean of the output patterns. Notice that when using weight decay, Hebb's rule and the covariance rule are no longer equivalent in case of a centered network. Also compare with the results without weight decay shown in Figure 7.

are shown in Figure 8a illustrating that except for Hebb's rule, all methods improve on the final 20 patterns compared to when no weight decay is used see as shown in Figure 7a. Notice that the equivalence of centered Hebb's rule and the centered covariance rule (see equation S6) no longer holds when a weight decay term is used, as the weights of the final weight matrix depend on the order of the presented data points now. Independent of whether a weight decay term is used or not, Hebb's rule is limited when an input pattern has to be associated with a zero target since the update is zero, and thus no learning happens. This is different for the covariance rule in which the mean values are also subtracted from the target values, allowing associations with zero target values to be learned. In this experiment the covariance rule achieves an even better performance than Hebbian descent on the final 20 patterns, which, however, comes at the cost of reduced performance for older patterns. Hebbian descent achieves optimal performance on the latest patterns with a rather small weight decay, and the performance is similar to that without weight decay, indicating that the implicit mechanism of forgetting does not profit much from weight decay. While gradient descent also improves on recent patterns, it has a worse

performance than Hebbian descent and the covariance rule over all of the last 100 patterns.

To investigate the effect of correlated data, we also performed the same experiments on associating 1000 binary random patterns (RAND) with 1000 patterns of the ADULT data set. The results are shown in Figure 8b and illustrate that in the presence of correlation, even with weight decay, none of the three alternative learning rules performs better than Hebbian descent on the final 20 or any of the final 100 patterns. We performed many more experiments with other data sets and activation functions and refer to the supplementary material, section F, Table S7, for the detailed results.

In most cases, Hebbian descent performs significantly better than the other methods, and in the remaining cases, its performance is close to the best. Gradient descent either reaches a similar performance as Hebbian descent or performs significantly worse. We did not expect weight decay to allow gradient descent to become better than Hebbian descent as it does not solve the problem of very small or zero values of the derivative of the activation function. While Hebb's rule is always significantly worse than all other methods, the covariance rule reaches a very good performance in case of binary random input and output patterns, as shown in Figure 8a. When the output data are binary random, the covariance rule reaches a similar performance as Hebbian descent, but in all other cases, the performance is significantly worse than Hebbian descent. As the problem with correlation or redundancy is still present for Hebb's rule and covariance rule, we did not expect that weight decay leads to a better performance than Hebbian descent. In Figure 9, we compare the performance of all methods with and without weight decay. All methods profit from using weight decay as all points lie below the diagonal. It is obvious to see that Hebbian descent has the best performance on average, closely followed by gradient descent, as the points are located much closer to the origin compared to the other methods. Gradient descent profits in most cases from weight decay, as shown in Figure 9a, although the improvement for Hebb's rule and the covariance rule is much higher, as shown in Figures 9c and 9d. Hebbian descent has a similar performance with and without weight decay in most cases, as shown in Figure 9b. In fact, in most of these cases, the optimal weight decay is even zero or at least very small (not shown). In case of unbounded activation functions, such as linear, rectifier, and exponential linear units, a weight decay term often has a slightly positive effect by keeping the weights from growing too large (i.e., overshooting). The use of a small weight decay term when using an unbounded activation function therefore seems to be a good idea in general.

5.1.5 Heteroassociative Multi-Epoch Learning. In contrast to Hebb's rule and the covariance rule, gradient descent and Hebbian descent take advantage of seeing training patterns several times. Repetitions help to stabilize memories (think of humans learning vocabularies, for example). This

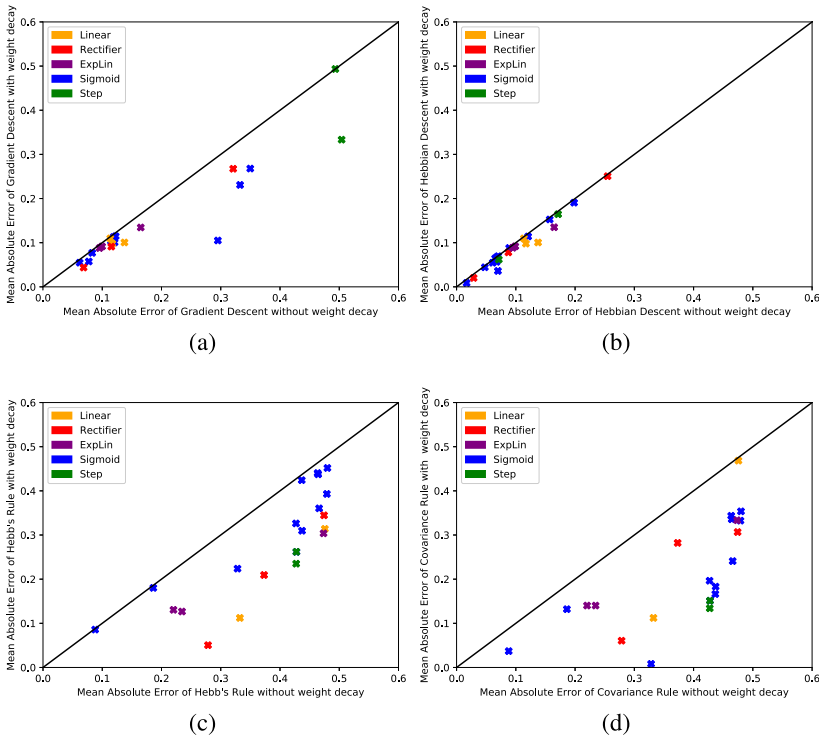


Figure 9: Comparison of online heteroassociation with and without weight decay when using centering and (a) gradient descent, (b) Hebbian descent, (c) Hebb's rule, and (d) covariance rule. Each cross represents the MAE of the final 20 patterns averaged over 10 trials for one experiment.

is a clear advantage as the methods can further improve the network's performance over time. We used the same training setup as in the continual learning experiments to train networks using gradient descent and Hebbian descent, but this time for 100 epochs instead of one. In other words, we continued training for another 99 epochs to investigate how much the networks can improve their performance when each pattern is presented several times. The results for centered and uncentered networks can be found in the supplementary material in section F, Tables S8, S13, and S14. We refer to Melchior (2021) for further experiments in the multi-epoch setting.

In all cases, whether centered or not, both methods improve significantly compared to when training only a single epoch. Furthermore, both methods reach better results when centering is used, and networks trained with centered Hebbian descent reach more often even a zero error value. In most cases, Hebbian descent reaches better values than gradient descent,

especially when the association can be learned almost perfectly as in the case of $\text{RAND} \rightarrow \text{ADULT}$, $\text{RANDN} \rightarrow \text{CONNECT}$, or $\text{MNIST} \rightarrow \text{CONNECT}$, for example. Both algorithms have a similar performance for exponential linear units on all data sets, but the performance of rectifier, sigmoid, or step function is more often better when using Hebbian descent. In case of the rectifier as an activation function, gradient descent often gets trapped in rather bad local optima, a direct consequence of the rectifier having a zero derivative for negative input values. As the derivative of the step function is constantly zero, only Hebbian descent can deal with the step function as a nonlinearity. The general tendency of Hebbian descent to perform better than gradient descent is even more prominent without centering.

To summarize, both methods reach similar results, but only Hebbian descent reaches good results also in continual learning (i.e., seeing every pattern only once). Similar results are also achieved when mini-batch learning (e.g., 10) is used instead of a batch size of 1.

5.2 Deep Network Experiments. In this section, we compare the classification and regression performance of Hebbian descent against gradient descent in deep neural networks. For classification results of single-layer networks, we refer to the supplementary material. In the classification setting, we present performance numbers on the MNIST and CIFAR data sets after five epochs of training. For the regression setting, we trained a denoising autoencoder on CIFAR-10 and report the final MAE after 80 epochs. Input images were treated with random perturbations in hue, saturation, value, and brightness of maximally 20% and had to be reconstructed to the original image. For a fair comparison, we chose per experiment the same data set, batch size, network architecture, and optimizer for both Hebbian descent and gradient descent. To accommodate for the difference in effective learning rates, we then performed a separate grid search for the optimal learning rate per learning rule. Figure 10a shows the final misclassification rate of Hebbian descent versus gradient descent in the classification setting, and Figure 10b shows the final mean absolute error of Hebbian descent versus gradient descent in the denoising autoencoder setting. The full experimental results are listed in the supplementary material in section F, Tables S2 and S3.

For the classification experiments (see Figure 10a), we see that Hebbian descent is either on par with or outperforms gradient descent for all experiments. This is especially the case when a convolutional architecture is applied to the more difficult CIFAR data set. While the lead is less pronounced for the higher-complexity ResNet18 architecture, as can also be seen in Table S2, Hebbian descent provides a measurable benefit over gradient descent as well, and this still holds if the state-of-the-art Adam optimizer is used. If the uncommon combination of MSE loss and linear output layer is used to represent Hebbian descent (not plotted here), the picture is less clear.

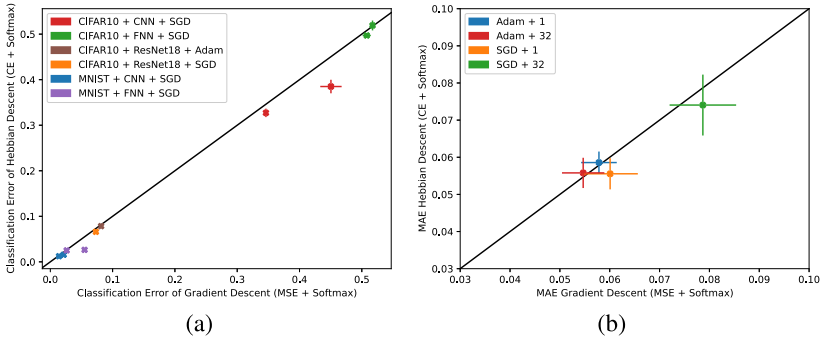


Figure 10: Comparison of Hebbian descent and gradient descent in deep networks. (a) Classification: Each cross represents the misclassification rate of a combination of network architecture, data set, and optimizer averaged over five trials. Hebbian descent represented by CE loss and softmax output-layer activation versus gradient descent using MSE loss and softmax output-layer activation. (b) Regression: Each cross represents the validation MAE of a denoising autoencoder using the combination of optimizer and batch size indicated in the legend. Hebbian descent is represented by binary CE loss and sigmoid output-layer activation versus gradient descent using MSE loss and sigmoid output-layer activation.

We still see a notable advantage for Hebbian descent in case of the simple convolutional architecture on CIFAR but slightly weaker performance in case of the simple feedforward architecture. The ResNet18 performs within a margin of error identical to the gradient descent setting.

In the autoencoder experiments (see Figure 10b), Hebbian descent performs better on average if the SGD optimizer is used. If Adam is used, gradient descent is slightly in the lead, whereas the performance difference between both is notably less pronounced than the one in the SGD experiments.

Our empirical results suggest that if nonsaturating activation functions are used on the hidden layers, Hebbian descent provides a useful perspective for deep network training in a way similar to that for shallow network training. Although it does not provide improved results across all error terms, architectures, data sets, batch sizes, and tasks, we achieved notably better or at least similar performance in the majority of experiments. Interestingly, even if we deviate from the best practice of using CE loss in conjunction with softmax in the classification experiments and pair MSE with linear activation instead, convolutional networks seem to either benefit from our alternative Hebbian descent error term or remain untouched performance-wise. While the choice of an unbounded activation function for learning one-hot vectors is suboptimal to begin with (see section 5.1.3), we nonetheless emphasize that Hebbian descent can provide tangible

benefits over gradient descent here even if the activation function does not perfectly match the criteria defined by the problem at hand.

6 Conclusion

In this study, we introduce Hebbian descent as a theoretical framework for avoiding negative effects of saturating output-layer activation functions in artificial neural networks, particularly in continual learning scenarios. By reviewing the relevant literature, we situated Hebbian descent within a lineage of similar concepts in section 1 and demonstrate that it unifies Hebbian learning, the covariance rule, and generalized linear models under a common principle (see section 3). The approach is operationalized by means of an alternative loss function for gradient descent, denoted as the Hebbian descent loss. This loss function is analogous to a weight update rule for the last layer of neural networks but with the derivative of the activation function removed. We demonstrate empirically that Hebbian descent can be used as a replacement for gradient descent as well as Hebbian learning as it inherits their advantages while not suffering from their disadvantages.

While in deep networks the hidden-layer activation function can usually be chosen as desired; for example, rectified linear units, employing saturating activation functions in the output layer, may be a necessity dictated by specific network applications. Here, Hebbian descent provides a complementary solution to nonsaturating hidden-layer activation functions, which mitigates the potentially negative effects of saturating activation functions in the output layer, where it would inevitably affect parameter updates in all previous layers.

In section 3.2, we argued that the major drawbacks of Hebbian learning are the problem of dealing with correlated or redundant input data and that it does not profit from seeing training patterns several times. For gradient descent, we identify the derivative of the final-layer activation function as problematic as it can lead to a vanishing error term that prevents efficient continual learning (see section 2). Hebbian descent addresses both of these problems. It exhibits the same convergence properties as gradient descent (provided that the derivative of the output-layer activation function is strictly positive), does not suffer from the vanishing error term problem, can deal with correlated data, profits from seeing patterns several times, and enables successful continual learning in single-layer networks when centering is used.

We show analytically that:

1. The Hebbian descent update can generally be understood as gradient descent update in which the derivative of the activation function of the output layer is removed (see section 2).
2. In case of a strictly positive derivative of the activation function in the output layer, Hebbian descent leads to the same update rule as

Table 1: Comparison of Gradient Descent (GD), Hebbian Learning/Covariance Rule (HL/CR), and Hebbian Descent (HD).

	GD	HL/CR	HD
Continual learning		+	+
Multi-epoch/batch learning	+		+
Convergent and stable	+		+
Profits from centering	+	+	+
Can deal with correlated patterns	+		+
No vanishing error term		+	+
No catastrophic interference		+	+
Inherent plausible forgetting mechanism			+

Notes: A plus sign indicates that the learning rule has the specified property. All properties are formulated so that a plus is advantageous.

gradient descent with a different loss function we name Hebbian descent loss (see equation 3.1). It thus inherits the convergence properties but even converges empirically when the derivative of the activation function is merely positive.

3. In the case of the mean squared error loss, Hebbian descent can be understood as the difference of a supervised and an unsupervised Hebbian learning step (see section 3.2), and with an invertible and integrable activation function Hebbian descent actually optimizes a generalized linear model (see section 3.3).

Our empirical results suggest that:

4. All update rules considered in this work profit from centering (see section 5.1.2).
5. Hebbian descent performs significantly better in continual learning than all other considered update rules (see section 5.1.2). Only Hebbian descent with centering shows an inherent and plausible curve of forgetting so that no additional forgetting mechanism like a weight decay term is required (see section 5.1.4).
6. In mini-batch learning for shallow and deep neural networks, the Hebbian descent loss performs better than or similar to the original loss with gradient descent (see section 5.2).

A comparison of the properties of Hebbian descent, gradient descent, and Hebbian learning rules is given in Table 1. The empirical evaluation of less conventional Hebbian descent losses (see appendix A) in deep neural networks is therefore an interesting future research direction. Future work might also focus on Hebbian descent learning in spiking neural networks since an implementation of Hebbian descent through spike-timing-dependent plasticity is obvious.

Appendix A: List of Hebbian Descent Loss Functions

Table 2 lists some Hebbian descent loss functions for different activation functions and error terms. As shown in section 3, using these loss functions with gradient descent leads to the Hebbian descent update. For a single

Table 2: List of Hebbian Descent Loss Functions (See Equation 3.1) for Various Activation Functions and Error Terms.

For $\mathcal{L}_{GD}(t_j, h_j) = \frac{1}{2} (h_j - t_j)^2$		
$h_j = \phi(a_j)$	$\frac{\mathcal{E}(t_j, h_j)}{\phi'(a_j)} = \frac{h_j - t_j}{\phi'(a_j)}$	$\mathcal{L}_{HD}(t_j, h_j) = \int \frac{\mathcal{E}(t_j, h_j)}{\phi'(a_j)} dh_j$
Linear		Squared error loss
a_j	$\frac{h_j - t_j}{1}$	$\frac{1}{2} (h_j - t_j)^2$
Sigmoid		Cross-entropy loss
$\frac{1}{1 + \exp(-a_j)}$	$\frac{h_j - t_j}{h_j(1 - h_j)}$	$-t_j \ln(h_j) - (1 - t_j) \ln(1 - h_j)$
Softmax		Cross-entropy loss
(Bridle, 1990) $\frac{\exp(a_j)}{\sum_k \exp(a_k)}$	$\frac{h_j - t_j}{h_j(\delta_{ij} - h_i)}$	$-\sum_j t_j \ln(h_j)$
Scaled Hyperbolic Tangent		
$\alpha \tanh(a_j)$	$\frac{h_j - t_j}{\alpha(1 - h_j^2)}$	$-\frac{1}{2\alpha} (1 + t_j) \ln(1 + h_j) - \frac{1}{2\alpha} (1 - t_j) \ln(1 - h_j)$
Approx. Step ($\alpha \rightarrow 0$)		
$\begin{cases} \alpha a_j \\ \alpha a_j + \beta \end{cases}$	$\begin{cases} \frac{h_j - t_j}{\alpha} \\ \frac{h_j - t_j}{1} \end{cases}$	$\begin{cases} \frac{1}{2\alpha} (h_j - t_j)^2 & \text{for } a_j < 0 \\ \frac{1}{2} (h_j - t_j)^2 & \text{for } a_j \geq 0 \end{cases}$
Leaky Rectifier. (Maas et al., 2013)		
$\begin{cases} \alpha a_j \\ a_j \end{cases}$	$\begin{cases} \frac{h_j - t_j}{\alpha} \\ \frac{h_j - t_j}{1} \end{cases}$	$\begin{cases} \frac{1}{2\alpha} (h_j - t_j)^2 & \text{for } a_j < 0 \\ \frac{1}{2} (h_j - t_j)^2 & \text{for } a_j \geq 0 \end{cases}$
Scaled Exp. Linear (Klambauer et al., 2017)		
$\begin{cases} \lambda \alpha (\exp(a_j) - 1) \\ \lambda a_j \end{cases}$	$\begin{cases} \frac{h_j - t_j}{h_j + \lambda \alpha} \\ \frac{h_j - t_j}{\lambda} \end{cases}$	$\begin{cases} h_j - (t_j + \lambda \alpha) \log(h_j + \lambda \alpha) & \text{for } a_j < 0 \\ \frac{1}{2\lambda} (h_j - t_j)^2 & \text{for } a_j \geq 0 \end{cases}$
Inv. Sqrt. (Carlile et al., 2017)		
$\frac{a_j}{\sqrt{1 + \alpha a_j^2}}$	$\frac{h_j - t_j}{(\alpha a_j^2 + 1)^{\frac{3}{2}}}$	$\frac{1}{2} (\alpha a_j^2 + 1)^{-\frac{3}{2}} (h_j - t_j)^2$

Table 2: Continued.

Inv. Sqrt. Linear (Carlile et al., 2017)		
$\begin{cases} \frac{a_j}{\sqrt{1+\alpha a_j^2}} \\ a_j \end{cases}$	$\begin{cases} \frac{h_j-t_j}{(\alpha a_j^2+1)^{-\frac{3}{2}}} \\ \frac{h_j-t_j}{1} \end{cases}$	$\begin{cases} \frac{1}{2} (\alpha a_j^2 + 1)^{\frac{3}{2}} (h_j - t_j)^2 & \text{for } a_j < 0 \\ \frac{1}{2} (h_j - t_j)^2 & \text{for } a_j \geq 0 \end{cases}$
SoftSign (Bergstra et al., 2009)		
$\frac{a_j}{1+ a_j }$	$\frac{h_j-t_j}{(1+ a_j)^{-2}}$	$\frac{1}{2}(1+ a_j)^2 (h_j - t_j)^2$
SoftPlus (Glorot et al., 2011)		
$\log(1 + \exp(a_j))$	$\frac{h_j-t_j}{(1+\exp(-a_j))^{-1}}$	$\frac{1}{2}(1 + \exp(-a_j)) (h_j - t_j)^2$
For $\mathcal{L}_{GD}(t_j, h_j) = \begin{cases} 1 - t_j h_j & \text{for } t_j h_j < 1 \\ \alpha h_j & \text{for } t_j h_j \geq 1 \end{cases}$		
$h_j = \phi(a_j)$	$(\alpha > 0, \alpha \rightarrow 0)$	$\mathcal{L}_{HD}(t_j, h_j) = \int \frac{\mathcal{E}(t_j, h_j)}{\phi'(a_j)} dh_j$
	$\frac{\mathcal{E}(t_j, h_j)}{\phi'(a_j)} = \begin{cases} \frac{-t_j}{\phi'(a_j)} \\ \frac{\alpha}{\phi'(a_j)} \end{cases}$	
Linear a_j		
	$(\alpha > 0, \alpha \rightarrow 0)$	Leaky version of the hinge loss
	$\begin{cases} \frac{-t_j}{1} \\ \frac{\alpha}{1} \end{cases}$	$\begin{cases} 1 - t_j h_j & \text{for } t_j h_j < 1 \\ \alpha h_j & \text{for } t_j h_j \geq 1 \end{cases}$
Sigmoid $\frac{1}{1+\exp(-a_j)}$		
	$(\alpha > 0, \alpha \rightarrow 0)$	
	$\begin{cases} \frac{-t_j}{h_j(1-h_j)} \\ \frac{\alpha}{h_j(1-h_j)} \end{cases}$	$\begin{cases} -t_j \ln(h_j) + t_j \ln(1 - h_j) & \text{for } t_j h_j < 1 \\ +\alpha \ln(h_j) - \alpha \ln(1 - h_j) & \text{for } t_j h_j \geq 1 \end{cases}$
For $\mathcal{L}_{GD}(t_j, h_j) = \frac{\alpha}{\beta} \ln(\cosh(\beta(h_j - t_j)))$		
$h_j = \phi(a_j)$	$\frac{\mathcal{E}(t_j, h_j)}{\phi'(a_j)} = \frac{\alpha \tanh(\beta(h_j - t_j))}{\phi'(a_j)}$	$\mathcal{L}_{HD}(t_j, h_j) = \int \frac{\mathcal{E}(t_j, h_j)}{\phi'(a_j)} dh_j$
Linear a_j		
	$\frac{\alpha \tanh(\beta(h_j - t_j))}{1}$	A smooth version of the Huber loss $\frac{\alpha}{\beta} \ln(\cosh(\beta(h_j - t_j)))$

Note: For notational simplicity, given for a single output, but for multiple outputs, simply use $\mathcal{L}_{HD}(\mathbf{t}, \mathbf{h}) = \sum_j \mathcal{L}_{HD}(t_j, h_j)$.

weight update, we have

$$\begin{aligned}
 \Delta_{GD} w_{ij} &\stackrel{(2.8)}{=} -\eta(x_i - \mu_i) \frac{\partial \mathcal{L}_{HD}(t_j, h_j)}{\partial h_j} \phi'(a_j) \\
 &\stackrel{(2.8)}{=} -\eta(x_i - \mu_i) \frac{\frac{\partial \mathcal{L}_{GD}(t_j, h_j)}{\partial h_j}}{\phi'(a_j)} \phi'(a_j) \\
 &\stackrel{(2.8)}{=} -\eta(x_i - \mu_i) \frac{\mathcal{E}(t_j, h_j)}{\phi'(a_j)} \phi'(a_j) \\
 &\stackrel{(2.8)}{=} -\eta(x_i - \mu_i) \mathcal{E}(t_j, h_j) \\
 &\stackrel{(3.3)}{=} \Delta_{HD} w_{ij}.
 \end{aligned}$$

In the following we give a detailed derivation of the derivative with respect to the preactivation $\mathbf{a}$ of the cross-entropy loss (Hebbian descent loss) with softmax units to show that it leads to the same gradient as the mean squared error loss with linear units:

$$h_j = \phi(a_j) = \frac{\exp(a_j)}{\sum_k \exp(a_k)}, \quad (\text{A.1})$$

$$h_j^{lin} = a_j, \quad (\text{A.2})$$

$$\mathcal{L}_{HD}(\mathbf{t}, \mathbf{h}) = - \sum_j t_j \ln(h_j), \quad (\text{A.3})$$

$$\mathcal{L}_{GD}(\mathbf{t}, \mathbf{h}) = \frac{1}{2} \sum_j (h_j - t_j)^2. \quad (\text{A.4})$$

The partial derivative of the softmax function with respect to a preactivation a_j is given by

$$\begin{aligned}
 \frac{\partial h_j}{\partial a_i} &= \frac{\partial}{\partial a_i} \phi(a_j) \stackrel{(\text{A.1})}{=} \frac{\partial}{\partial a_i} \frac{\exp(a_j)}{\sum_k \exp(a_k)} \\
 &= \delta_{ij} \frac{\exp(a_i)}{\sum_k \exp(a_k)} + \exp(a_j) \frac{\partial}{\partial a_i} \left(\sum_k \exp(a_k) \right)^{-1} \\
 &\stackrel{(\text{A.1})}{=} \delta_{ij} h_j - \exp(a_j) \exp(a_i) \left(\sum_k \exp(a_k) \right)^{-2} \\
 &= \delta_{ij} h_j - \frac{\exp(a_j)}{\sum_k \exp(a_k)} \frac{\exp(a_i)}{\sum_k \exp(a_k)}
 \end{aligned}$$

$$\begin{aligned}
&\stackrel{(A.1)}{=} \delta_{ij} h_j - h_j h_i \\
&= h_j (\delta_{ij} - h_i).
\end{aligned} \tag{A.5}$$

The partial derivative of the cross-entropy loss and softmax units with respect to a preactivation a_j is given by

$$\begin{aligned}
\frac{\partial \mathcal{L}_{HD}(\mathbf{t}, \mathbf{h})}{\partial a_i} &\stackrel{(A.4)}{=} \frac{\partial}{\partial a_i} \left(- \sum_j t_j \ln(h_j) \right) \\
&= - \sum_j t_j \frac{\partial}{\partial a_i} \ln(h_j) \\
&= - \sum_j t_j \frac{1}{h_j} \frac{\partial h_j}{\partial a_i} \\
&\stackrel{(A.5)}{=} - \sum_j t_j \frac{1}{h_j} h_j (\delta_{ij} - h_i) \\
&= - \sum_j t_j (\delta_{ij} - h_i) \\
&= \sum_j t_j h_i - \sum_j t_j \delta_{ij} \\
&= h_i \sum_j t_j - t_i \\
&= h_i - t_i.
\end{aligned} \tag{A.6}$$

The partial derivative of the mean squared error loss and linear units with respect to a preactivation a_j is given by

$$\frac{\partial \mathcal{L}_{GD}(\mathbf{t}, \mathbf{h}^{lin})}{\partial a_i} \stackrel{(A.3)}{=} \frac{\partial \frac{1}{2} \sum_j (h_j^{lin} - t_j)^2}{\partial a_i} \tag{A.7}$$

$$= h_i - t_i, \tag{A.8}$$

which is indeed the same as the partial derivative of the cross entropy loss and softmax units with respect to a preactivation a_j .

This illustrates the simplicity of Hebbian descent where we do not need to calculate the derivatives of the activation function at all, instead using the mean squared error loss and drop the derivatives of the activation function in the calculation of the gradient.

Appendix B: Hebbian Descent Loss Is the General Log-Likelihood Loss

In the following, we illustrate the equality between the Hebbian descent and the general log-likelihood loss in case of a sigmoid activation function $\phi(a_j) = (1 + \exp(-a_j))^{-1}$. In this case, the link function is given as the inverse of the sigmoid by $\psi(\gamma_j) = \log(\gamma_j(1 - \gamma_j))^{-1}$, and the corresponding antiderivative becomes $\int \phi(a_j) da_j = \log(1 + \exp(a_j))$. One can now show that for $b(t_j) = 1$, this choice leads to the Bernoulli distribution, which is usually defined with respect to the expectation value of output $\gamma_j = E[t_j]$ as follows:

$$p(t_j | \mathbf{x}, \mathbf{W}, \mathbf{b}) = \exp(a_j t_j - \log(1 + \exp(a_j))) \quad (\text{B.1})$$

$$\stackrel{(3.16)}{=} \exp(\psi(\gamma_j))^{t_j} (1 + \exp(\psi(\gamma_j)))^{-1} \quad (\text{B.2})$$

$$= \exp\left(\log(\gamma_j(1 - \gamma_j)^{-1})\right)^{t_j} \left(1 + \exp\left(\log(\gamma_j(1 - \gamma_j)^{-1})\right)\right)^{-1} \quad (\text{B.3})$$

$$= (\gamma_j(1 - \gamma_j)^{-1})^{t_j} (1 + \gamma_j(1 - \gamma_j)^{-1})^{-1} \quad (\text{B.4})$$

$$= \gamma_j^{t_j} (1 - \gamma_j)^{-t_j} ((1 - \gamma_j)(1 - \gamma_j)^{-1} + \gamma_j(1 - \gamma_j)^{-1})^{-1} \quad (\text{B.5})$$

$$= \gamma_j^{t_j} (1 - \gamma_j)^{-t_j} (1 - \gamma_j + \gamma_j)(1 - \gamma_j)^{-1})^{-1} \quad (\text{B.6})$$

$$= \gamma_j^{t_j} (1 - \gamma_j)^{-t_j} (1 - \gamma_j) \quad (\text{B.7})$$

$$= \gamma_j^{t_j} (1 - \gamma_j)^{1-t_j}, \quad (\text{B.8})$$

which is the Bernoulli-distribution for the output variable t_j taking the value 1 with probability γ_j and the value zero with probability $(1 - \gamma_j)$.

Acknowledgments

We thank Amir Hossein Azizi and Mehdi Bayati for helpful discussions on Hebbian learning.

References

- Ans, B., & Rousset, S. (1997). Avoiding catastrophic forgetting by coupling two reverberating neural networks. *Comptes Rendus de l'Académie des Sciences, Series III: Sciences de la Vie*, 320(12), 989–997.

- Bergstra, J., Desjardins, G., Lamblin, P., & Bengio, Y. (2009). *Quadratic polynomials learn better image features*. Technical report 1337. Département d'Informatique et de Recherche Opérationnelle, Université de Montréal.
- Biehl, M., & Schwarze, H. (1995). Learning by on-line gradient descent. *Journal of Physics A: Mathematical and General*, 28(3), 643. 10.1088/0305-4470/28/3/018
- Bienenstock, E. L., Cooper, L. N., & Munro, P. W. (1982). Theory for the development of neuron selectivity: Orientation specificity and binocular interaction in visual cortex. *Journal of Neuroscience*, 2(1), 32–48. 10.1523/JNEUROSCI.02-01-00032.1982
- Bridle, J. S. (1990). Probabilistic interpretation of feedforward classification network outputs, with relationships to statistical pattern recognition. In F. F. Soulié & J. Héroult (Eds.), *Neurocomputing* (pp. 227–236). Springer.
- Carlile, B., Delamarter, G., Kinney, P., Marti, A., & Whitney, B. (2017). *Improving deep learning by inverse square root linear units (ISRLUs)*. arXiv:1710.09967.
- Chen, J. (1990). Stepsize variation methods for accelerating the back propagation algorithm. In *Proceedings of the International Joint Conference on Neural Networks* (vol. 1, pp. 601–604).
- Cho, K., Raiko, T., & Ilin, A. (2010). Parallel tempering is efficient for learning restricted Boltzmann machines. In *Proceedings of the International Joint Conference on Neural Networks* (pp. 3246–3253).
- Clevert, D., Unterthiner, T., & Hochreiter, S. (2015). *Fast and accurate deep network learning by exponential linear units (ELUS)*. arXiv:1511.07289.
- Desjardins, G., Courville, A., Bengio, Y., Vincent, P., & Delalleau, O. (2010). Parallel tempering for training of restricted Boltzmann machines. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*.
- Duchi, J., Hazan, E., & Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(7).
- Eiter, T., & Kern-Isberner, G. (2019). A brief survey on forgetting from a knowledge representation and reasoning perspective. In *Künstliche Intelligenz*, 33(1), 9–33.
- Fahlman, S. E. (1988). Faster learning variations of back propagation: An empirical study. In *Proceedings of the 1988 Connectionist Models Summer School* (pp. 38–51).
- French, R. M. (1991). Catastrophic forgetting in connectionist networks. *Encyclopedia of Cognitive Science*.
- French, R. M. (1997). Pseudo-recurrent connectionist networks: An approach to the “sensitivity-stability” dilemma. *Connection Science*, 9(4), 353–380. 10.1080/095400997116595
- Fukushima, K., & Miyake, S. (1982). Neocognitron: A self-organizing neural network model for a mechanism of visual pattern recognition. In S.-I. Amari & M. A. Arbib (Eds.), *Competition and cooperation in neural nets* (pp. 267–285). Springer.
- Gallant, S. I. (1990). Perceptron-based learning algorithms. *IEEE Transactions on neural networks*, 1(2), 179–191. 10.1109/72.80230
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feed-forward neural networks. In *Proceedings of the International Conference on Artificial Intelligence and Statistics*.
- Glorot, X., Bordes, A., & Bengio, Y. (2011). Deep sparse rectifier neural networks. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics* (pp. 315–323).

- Goel, S., Klivans, A., & Meka, R. (2018). Learning one convolutional layer with overlapping patches. In J. Dy & A. Krause (Eds.), *In Proceedings of the 35th International Conference on Machine Learning* (pp. 1783–1791).
- Hahnloser, R. H. R., Sarpeshkar, R., Mahowald, M. A., Douglas, R. J., & Seung, H. S. (2000). Digital selection and analogue amplification coexist in a cortex-inspired silicon circuit. *Nature*, 405(6789), 947. 10.1038/35016072
- Hahnloser, R. H. R., & Seung, H. S. (2001). Permitted and forbidden sets in symmetric threshold-linear networks. In T. G. Dietterich, S. Becker, & Z. Ghahramani (Eds.), *Advances in neural information processing systems*, 14 (pp. 217–223). Cambridge, MA: MIT Press.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*.
- Hebb, D. O. (1949). *The organization of behavior: A neuropsychological approach*. Wiley.
- Hertz, J., Krogh, A., Lautrup, B., & Lehmann, T. (1997). Nonlinear backpropagation: Doing backpropagation without derivatives of the activation function. *IEEE Transactions on Neural Networks*, 8(6), 1321–1327. 10.1109/72.641455
- Hinton, G. E. (1989). Connectionist learning procedures. *Artificial Intelligence*, 40(1–3), 185–234. 10.1016/0004-3702(89)90049-0
- Hinton, G. E. (2002). Training products of experts by minimizing contrastive divergence. *Neural Computation*, 14, 1771–1800. 10.1162/089976602760128018
- Hochreiter, S. (1991). *Untersuchungen zu dynamischen neuronalen netzen*. Diploma, Technische Universität München.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. 10.1162/neco.1997.9.8.1735
- Ioffe, S., & Szegedy, C. (2015). *Batch normalization: Accelerating deep network training by reducing internal covariate shift*. arXiv:1502.03167.
- Kakade, S. M., Kanade, V., Shamir, O., & Kalai, A. (2011). Efficient learning of generalized linear and single index models with isotonic regression. In J. Shawe-Taylor, R. Zemel, Bartlett, F. Pereira, & K. Q. Weinberger (Eds.), *Advances in neural information processing systems*, 24 (pp. 927–935). Curran.
- Kelley, H. J. (1960). Gradient theory of optimal flight paths. *ARS Journal*, 30(10), 947–954. 10.2514/8.5282
- Kingma, D. P., & Ba, J. (2014). *Adam: A method for stochastic optimization*. arXiv: 1412.6980.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., . . . Hadseli, R. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13), 3521–3526. 10.1073/pnas.1611835114
- Klambauer, G., Unterthiner, T., Mayr, A., & Hochreiter, S. (2017). Self-normalizing neural networks. In I. Guyon, Y. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in neural information processing systems*, 30 (pp. 972–981). Curran.
- Krizhevsky, A. (2009). Learning multiple layers of features from tiny images. Master's thesis, University of Toronto.
- Kumar, S., Walia, S., & Kalra, A. (2015). ANN training: A review of soft computing approaches. *International Journal of Electrical and Electronics Engineering*, 2, 193–205. 10.14445/23488379/IJEEE-V2I11P104

- Larochelle, H., Bengio, Y., & Turian, J. (2010). Tractable multivariate binary density estimation and the restricted Boltzmann forest. *Neural Computation*, 22, 2285–2307. 10.1162/NECO_a_00014
- LeCun, Y. A., Bottou, L., Orr, G. B., & Müller, K.-R. (2012). Efficient BackProp. In G. Montavon, G. B. Orr, & K. R. Müller (Eds.), *Neural networks: Tricks of the trade* (pp. 9–48). Lecture Notes in Computer Science, vol. 7700. Springer. 10.1007/978-3-642-35289-8_3
- Lee, Y., Oh, S.-H., & Kim, M. W. (1993). An analysis of premature saturation in back propagation learning. *Neural Networks*, 6(5), 719–728. 10.1016/S0893-6080(05)80116-9
- Löwe, M. (1998). On the storage capacity of Hopfield models with correlated patterns. *Annals of Applied Probability*, 8(4), 1216–1250.
- Maas, A. L., Hannun, A. Y., & Ng, A. Y. (2013). Rectifier nonlinearities improve neural network acoustic models. In *Proceedings of the International Conference on Machine Learning*.
- Marr, D., Willshaw, D., & McNaughton, B. (1991). Simple memory: A theory for archicortex. In L. Vaina (Ed.), *From the retina to the neocortex* (pp. 59–128). Springer.
- McClelland, J. L., McNaughton, B. L., & O'Reilly, R. C. (1995). Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3), 419. 10.1037/0033-295X.102.3.419
- McCloskey, M., & Cohen, N. J. (1989). Catastrophic interference in connectionist networks: The sequential learning problem. In B. H. Ross (Ed.), *Psychology of learning and motivation* (pp. 109–165). Elsevier.
- McCullagh, P., & Nelder, J. A. (1989). *Generalized linear models*. CRC Press.
- Melchior, J. (2021). *On the importance of centering in artificial neural networks*. PhD diss., Ruhr-Universität Bochum.
- Melchior, J., Fischer, A., & Wiskott, L. (2016). How to center deep Boltzmann machines. *Journal of Machine Learning Research*, 17(99), 1–61.
- Mikolov, T., Joulin, A., Chopra, S., Mathieu, M., & Ranzato, M. (2014). *Learning longer memory in recurrent neural networks*. arXiv:1412.7753.
- Montavon, G., & Müller, K. R. (2012). Deep Boltzmann machines and the centering trick. *Lecture Notes in Computer Science*, 7700 (pp. 621–637). Springer. 10.1007/978-3-642-35289-8_33
- Movellan, J. R. (1991). Contrastive Hebbian learning in the continuous Hopfield model. In D. S. Touretzky, J. L. Elman, T. Sejnowski, & G. E. Hinton (Eds.), *Connectionist models* (pp. 10–17). Elsevier.
- Neher, T., Cheng, S., & Wiskott, L. (2015). Memory storage fidelity in the hippocampal circuit: The role of subregions and input statistics. *PLOS Computational Biology*, 11(5), e1004250. 10.1371/journal.pcbi.1004250
- Nelder, J. A., & Baker, R. J. (1972). *Generalized linear models*. Wiley.
- Ng, S., Cheung, C., Leung, S., & Luk, A. (2003). Fast convergence for backpropagation network with magnified gradient function. In *Proceedings of the International Joint Conference on Neural Networks*, 2003 (pp. 1903–1908).
- Ng, S. C., Leung, S. H., & Luk, A. (1999). Fast convergent generalized backpropagation algorithm with constant learning rate. *Neural Processing Letters*, 9, 13–23. 10.1023/A:1018611626332

- Oja, E. (1982). Simplified neuron model as a principal component analyzer. *Journal of Mathematical Biology*, 15(3), 267–273. 10.1007/BF00275687
- Ooyen, A., & Nienhuis, B. (1992). Improving the convergence of the backpropagation algorithm. *Neural Networks*, 5, 465–471. 10.1016/0893-6080(92)90008-7
- Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning*.
- Ratcliff, R. (1990). Connectionist models of recognition memory: Constraints imposed by learning and forgetting functions. *Psychological Review*, 97(2), 285. 10.1037/0033-295X.97.2.285
- Riedmiller, M., & Braun, H. (1993). A direct adaptive method for faster backpropagation learning: The RPROP algorithm. In *Proceedings of the IEEE International Conference on Neural Networks* (pp. 586–591).
- Robbins, H., & Siegmund, D. (1985). A convergence theorem for non negative almost supermartingales and some applications. In T. Lai & D. Siegmund (Eds.), *Herbert Robbins selected papers* (pp. 111–135). Springer.
- Robins, A. (1995). Catastrophic forgetting, rehearsal and pseudorehearsal. *Connection Science*, 7(2), 123–146. 10.1080/09540099550039318
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408. 10.1037/h0042519
- Rumelhart, D., Hinton, G. E., & Williams, R. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536. 10.1038/323533a0
- Rumelhart, D., McClelland, J., & the PDP Research Group. (1986). *Parallel distributed processing: Explorations in the microstructure of cognition*. MIT Press.
- Saad, D. (1998). Online algorithms and stochastic approximations. *Online Learning*, 5.
- Salehinejad, H., Sankar, S., Barfett, J., Colak, E., & Valaee, S. (2018). *Recent advances in recurrent neural networks*. arXiv:abs/1801.01078.
- Sanger, T. D. (1989). Optimal unsupervised learning in a single-layer linear feed-forward neural network. *Neural Networks*, 2(6), 459–473. 10.1016/0893-6080(89)90044-0
- Santoro, A., Bartunov, S., Botvinick, M., Wierstra, D., & Lillicrap, T. (2016). *One-shot learning with memory-augmented neural networks*. arXiv:1605.06065
- Schraudolph, N. (1998). Centering neural network gradient factors. In G. Montavon, G. Orr, and K.-R. Müller (Eds.), *Neural networks: Tricks of the trade*. Lecture Notes in Computer Science, (pp. 207–226). Springer.
- Sejnowski, T. J., & Tesauero, G. (1989). The Hebb rule for synaptic plasticity: Algorithms and implementations. In J. H. B. Byrne (Ed.), *Neural models of plasticity: Experimental and theoretical approaches* (pp. 94–103). Elsevier
- Tieleman, T. (2008). Training restricted Boltzmann machines using approximations to the likelihood gradient. In *Proceedings of the International Conference on Machine Learning* (pp. 1064–1071).
- Vitela, J. E., & Reifman, J. (1993). Enhanced backpropagation training algorithm for transient event identification. In *Transactions of the American Nuclear Society*, 69. ANS.

- Vora, K., & Yagnik, S. B. (2013). A survey on backpropagation algorithms for feed-forward neural networks. *International Journal of Engineering Development and Research*, 1(3), 193–197.
- Widrow, B., & Hoff, M. E. (1960). Adaptive switching circuits. In *1960 IRE WESCON Convention Record, Part 4* (pp. 96–104).
- Xie, X., & Seung, H. S. (2003). Equivalence of backpropagation and contrastive Hebbian learning in a layered network. *Neural Computation*, 15(2), 441–454. 10.1162/089976603762552988
- Yu, C.-C., Tang, Y.-C., & Liu, B.-D. (2002). An adaptive activation function for multilayer feedforward neural networks. In *Proceedings of the 2002 IEEE Region 10 Conference on Computers, Communications, Control and Power Engineering* (pp. 645–650).

Received January 4, 2023; accepted March 27, 2024.

Copyright of Neural Computation is the property of MIT Press and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.